

AutoQuREO: A Framework for Automated Quantum Resource Estimation and Optimization

Harshkumar Oza ^{*}, Aritra Sarkar ^{*}, Syed Naqi Abbas , Rahul Bhowmick ,
Aryan Prakash , Prateek P Kulkarni , Krishna Kumar Sabapathy 

Quantum Lab, Fujitsu Research of India

^{*}Equal contribution.

As quantum computing progresses from proof-of-principle demonstrations toward practical utility, a significant impediment is the need to augment algorithmic feasibility with system-level optimization across heterogeneous hardware and software stacks. Quantum resource estimation (QRE) plays a central role in this transition, yet existing approaches remain largely compilation-heavy or domain-knowledge-guided symbolic annotations, and tightly coupled to long-term fault-tolerant assumptions, limiting their topical applicability.

In this work, we introduce AutoQuREO, an Automated framework for full-stack Quantum Resource Estimation and Optimization. AutoQuREO is built around four core novelties: (i) a flexible, user-defined abstraction of the quantum computing stack; (ii) a modular library of reusable stack components enabling rapid full-stack prototyping; (iii) surrogate modeling of layer-wise resources via algorithmic profiling and neuro-symbolic learning; and (iv) integrated multi-objective optimization that embeds QRE directly into deployment pipelines. Together, these design choices enable AutoQuREO to serve as a digital twin for quantum computing stacks, supporting the tractable exploration of complex design spaces.

We demonstrate the capabilities of AutoQuREO through representative co-design case studies, including early-fault-tolerant quantum algorithms, small error correction codes, gate decomposition and variational training of parametric quantum circuits. These examples illustrate how AutoQuREO enables systematic discovery of unexploited resource trade-offs that are computationally intractable or abstruse using existing QRE tools. AutoQuREO is positioned as a general-purpose platform for advancing quantum technology readiness.

 **Date:** August 14, 2026

 **Correspondence:** Aritra Sarkar at aritra.sarkar@fujitsu.com

Contents

1	Introduction	2
2	Background and related works	3
2.1	Quantum resource estimation	3
2.2	Quantum algorithmic profiling	5
3	The AutoQuREO framework	6
3.1	Design principles and novelty	6
3.2	Layer definition	6
3.3	Adapters	7
3.4	Resource models	9
3.5	Adapter metadata and arbitration	10
3.6	Surrogate synthesis	10
3.7	Lifelong learning	10
3.8	Resource analysis and optimization	12
3.9	Workflow of a QRE scenario	13
3.10	Software architecture and execution flow	13
4	Exemplary co-design use cases	18
4.1	Scenario I: Trotterized Hamiltonian simulation and error mitigation	18
4.2	Scenario II	23
4.2.1	Universal error correction and connectivity topology	23
4.2.2	Gate decomposition accuracy and logical error rate	25
4.2.3	Ground-state energy estimation with iterative quantum phase estimation	30
4.2.4	Extension to fault-tolerant regimes	34
4.3	Scenario III: Parameterized circuits and hardware constraints for optimization	36
5	Discussion	40

1 Introduction

Quantum computing has witnessed rapid progress over the past decade, marked by improvements in hardware scale, fidelity, and system integration. On the algorithmic side, the complexity-theoretic quantum advantage [1] through problems in the bounded-error quantum polynomial time (BQP) class continues to motivate provable long-term speedups. This is exemplified by algorithms for factoring, simulating quantum many-body systems, and performing amplitude amplification. In parallel, a growing focus of heuristic and application-driven algorithms targets near-term and early fault-tolerant (EFT) [2] regimes. These developments have been accompanied by experimental demonstrations [3] of quantum computational supremacy in restricted sampling tasks, while a complementary line of work [4] focuses on scalable quantum error correction (QEC) as a principled path toward fault-tolerant quantum computation (FTQC).

To manage the complexity of system design, the quantum computing community has increasingly adopted a layered abstraction of the quantum computing stack [5], enabling a separation of concerns across algorithms, compilation, error correction, and hardware execution. The stack has proven invaluable for structuring research and engineering workflows, allowing specialists to focus on individual layers without being overwhelmed by system-wide details. However, as an inherent drawback of the approach, design choices made at one layer can substantially affect the feasibility of other layers, and thereby of the full system.

Hardware-software co-design is therefore increasingly gaining traction for achieving early quantum advantage [6, 7]. Co-design considers inter-layer trade-offs that snowball into system-level efficiency [8], particularly in the EFT regime. As a result, optimization objectives shift from studying asymptotic resource measures to multi-dimensional resource trade-offs involving qubits, depth, fidelity, and classical control cost, etc. This underscores the need for systematic tools that can reason across stack boundaries.

Exploring such trade-offs analytically is non-trivial and requires system-wide expertise. This motivates the development of quantum resource estimation (QRE) tools within a quantum software development kit (QSDK). QRE is intended to operate as a digital twin of a quantum system, estimating relevant resources without explicitly simulating the computation. This distinction is crucial as simulation quickly becomes intractable beyond modest system sizes, while resource estimation can remain scalable by disregarding state-level dynamics in favor of cost models. The utility of QRE extends beyond offline analysis and benchmarking. In quantum processing unit (QPU) execution pipelines, architectural insights from optimized small-scale configurations can inform design choices for scalable deployments. QRE can thus be embedded within iterative development workflows, becoming a tool for knowledge transfer across system scaling. Several QRE frameworks [9, 10, 11] have already been proposed and adopted by the community, recognizing QRE as a critical component of quantum software ecosystems. Prominent examples include Microsoft’s Azure Quantum Resource Estimator [11, 12], Google’s Qualtran [13], PsiQuantum’s Bartiq [14], Rigetti’s Resource Estimator (RRE) [15], QRE within Xanadu’s PennyLane compiler [16], Infleqtion’s resource-superstaq [17], QRE within Qrisp compiler [18] jointly developed with multiple European academic and industry partners, and the TopQAD platform [19] developed jointly by 1QBit, NVIDIA, and Synopsys, among others. These tools enable large-scale analyses of fault-tolerant quantum algorithms to set expectations for long-term quantum advantage. Consequently, the design choices of these tools largely reflect a fault-tolerant, algorithm-centric perspective. Resource estimation is anchored at the quantum algorithm layer, while downstream layers, such as gate decomposition, routing, error correction, and hardware configurations, are implicitly assumed based on an FTQC roadmap. As a result, these platforms provide limited support for optimizing downstream layers or for benchmarking a new method. Moreover, the QRE approaches rely heavily on either compilation-based resource extraction, which, while accurate, is computationally intractable for large-scale design space exploration (DSE), or on user-provided symbolic annotations that require a high level of cross-domain expertise. These limitations motivate the need for a more flexible and scalable QRE approach that remains applicable across NISQ, EFT, and FTQC regimes.

In this work, we introduce AutoQuREO, an automated quantum resource estimation and optimization framework designed to address these challenges. A visual overview of a typical AutoQuREO workflow is shown in Figure 1. AutoQuREO is distinguished by four core contributions. First, it provides full-stack flexibility, allowing users to define arbitrary layers, corresponding hyperparameters, and resource metrics of interest, rather than restricting DSE to a fixed FTQC stack. Second, it offers an extensive library of reusable code components and resource models for exemplary stack layers, reducing the barrier to full-stack QRE research. Third, it introduces surrogate modeling of layer-wise resources via algorithmic profiling and neuro-symbolic learning, enabling scalable estimation that progressively replaces costly compilation-based ground-truth while remaining analytically explainable. Finally, AutoQuREO embeds automated multi-objective optimization directly into the deployment pipeline, supporting Pareto analysis and informed selection of optimal configurations under competing constraints.

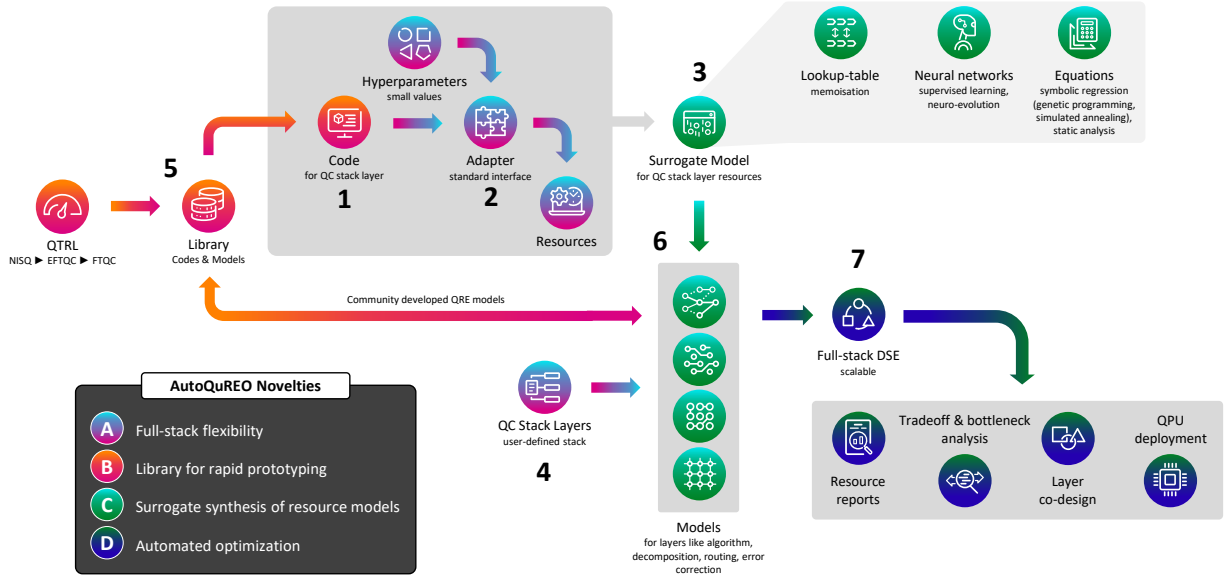


Figure 1 Overview of a typical AutoQuREO workflow. (1) Layer-specific code is either imported from the rapid-prototyping library or provided by the user. The library supports NISQ, EFTQC, and FTQC primitives. (2) The adapter design pattern interfaces the code, mapping hyperparameters to resources. The code is compiled for small instances to generate data for modeling. (3) A surrogate resource model is created. AutoQuREO natively supports various models and modeling techniques shown in the inset. (4) Multiple layers can be flexibly added by the user from the library to create a full-stack scenario. (5) The models can be imported from the library, or newly created models can be added back to the library for reuse and community development. (6) The models of the QC stack layers of algorithm, decomposition, error correction, routing, etc., define the project. (7) Parameters and resources of the full-stack can be co-designed via scalable and interpretable DSE. The optimal full-stack can be analyzed to understand bottlenecks and deployed for the target backend.

We demonstrate the utility of AutoQuREO’s salient features through three representative case studies: (i) co-design of Trotter-order and steps in Hamiltonian simulation with error-mitigated noisy hardware, (ii) co-design of a universal quantum error correction scheme, connectivity topology, gate decomposition accuracy, and early fault-tolerant algorithm for ground-state estimation on noisy hardware, and (iii) co-design of mixing ansätze, error mitigation, and routing for variational quantum optimization. Together, these examples illustrate how automated, full-stack quantum resource estimation can help navigate intricate design spaces to optimize system-level performance via surrogate models.

The remainder of this article is organized as follows. Section 2 reviews background concepts and related work on quantum computing stacks, resource estimation, and algorithmic profiling. In Section 3, we describe the design and architecture of the proposed AutoQuREO framework. Section 4 presents four detailed case studies demonstrating AutoQuREO’s capabilities. Section 5 concludes the article with a discussion of broader implications, limitations, and future directions.

2 Background and related works

This section reviews the conceptual and technical foundations underlying quantum resource estimation and algorithmic profiling, situating AutoQuREO within the broader landscape of quantum software tooling. We first summarize how resources are defined, propagated, and estimated across the quantum computing stack, and survey existing frameworks developed for this purpose. We then introduce algorithmic profiling as an empirical and interpretable approach to modeling resource costs, highlighting its relevance for scalable, full-stack quantum resource estimation.

2.1 Quantum resource estimation

Quantum advantage is ultimately a statement about resources rather than computability. The physical Church-Turing thesis (pCTT) provides the underlying foundation of computability in computer science, allowing any

physical system to be simulated by a universal computer. However, in view of pCTT, both classical and quantum computers (via their automata models, such as the universal Turing machine and the quantum Turing machine [20]) are equivalent in the Turing hierarchy, i.e., anything that can be computed by one can also be computed by the other. The separation between classical and quantum computation arises from the complexity-theoretic version [1] of the thesis, which positions quantum computation (QC) as a superset of classical computation (CC) when resources are taken into account. In view of that, all CC can be emulated with QC efficiency (i.e., with a worst-case polynomial overhead), for example, using gates like Toffoli that map to universal CC gates like NAND and FanOut. On the other hand, while all QC can also be emulated with CC via unitary evolution, for the general case, the time and memory cost grow exponentially in the system size. Thus, the quantum advantage boundary requires careful understanding of the inherent complexity in the problem formulation that is not amenable to efficient CC, as well as the exact resources of CC and QC for problem sizes of relevance in addition to asymptotics. Even provably asymptotic quantum speedups may be rendered irrelevant by prohibitive constant factors or fault-tolerance overheads. Conversely, heuristic or approximate algorithms without asymptotic guarantees may offer practical advantages when resource trade-offs align favorably. Quantum computational resources [21, 22, 23] thus play a pivotal role in translating abstract algorithmic promise into engineered performance.

Quantum resource estimation (QRE) concerns the systematic quantification of the dependencies between the physical costs required to realize a quantum computation on a given execution stack and the various configurations in the design space. Typical resources include the number of logical and physical qubits (space), circuit depth and wall-clock runtime (time), structured gate counts (e.g., Clifford vs non-Clifford), composite measures (e.g., active volume), approximation (e.g., target precision, logical error rates), and hardware metrics (e.g., connectivity, gate latency, classical control bandwidth, noise). Importantly, these resources are often not analyzable independently. For example, reductions in circuit depth may require additional qubits, while higher precision or fault tolerance can amplify space-time costs through error-correction overhead. Additionally, what constitutes a resource as input or output of the estimation or optimization is guided by the problem statement posed by the use case. For example, a user may either optimize an algorithm within the qubit limits of a specific hardware platform or study the qubit-scaling behavior of the algorithm with respect to problem size.

Historically, large-scale QRE has been carried out through painstaking analytical derivations. Seminal works such as [24, 25, 8, 26] provide detailed end-to-end resource analyses, requiring substantial effort from domain experts. While these studies set important benchmarks, their methodology does not scale to the diversity of quantum algorithms, hardware platforms, or error-correction schemes under active investigation. Moreover, such analyses are brittle in incorporating design choices; for instance, replacing surface codes with holographic codes [27] can invalidate substantial portions of the analysis. This limits their utility for rapid iteration, exploratory research, and cross-layer co-design.

The need for systematic tooling motivated some of the earliest QRE frameworks, including QuRE [28], which formalized resource accounting at the architectural level. This direction has since been reinforced by large-scale initiatives such as DARPA’s Quantum Benchmarking program that led to the development of tools such as pyLIQTR from MIT Lincoln Laboratory [29], Rigetti’s resource estimators [30], Zapata AI’s BenchQ [31], and academic tools such as Jabalizer [32] exemplify interest in software tools for resource analysis. QRE tools under active development have matured significantly, with four platforms standing out for adoption and scope. Microsoft’s Azure Quantum Resource Estimator (AQRE) [11, 12] provides symbolic and compilation-based estimation tightly integrated with fault-tolerant assumptions. Google’s Qualtran [13] introduces a structured, compositional language for expressing quantum algorithms as resource-annotated blocks (Bloqs) and symbolic surface code QEC models. PsiQuantum’s Bartiq and QREF focus on symbolic estimation of the resource requirements of algorithms aimed at the FTQC era. TopQAD [19] integrates algorithm-architecture co-design across partners, including IQBit, NVIDIA, and Synopsys. It includes rigorous assessments of FTQC surface code superconducting processors.

QRE tools are increasingly used to support scientific and engineering studies. For example, [12] uses AQRE to assess the feasibility of fault-tolerant implementations of chemistry, optimization, and simulation workloads. Such studies demonstrate the value of automated QRE for grounding algorithmic proposals in realistic resource budgets [33, 34] and for exploring alternative design pathways.

Despite significant progress, existing QRE frameworks share several topical limitations. Most are anchored to a fault-tolerant quantum computing (FTQC) roadmap [35], offering limited support for near-term or EFT research, characterized by experimenting with design choices across stack layers. By weakly parameterizing the lower stack layers to the FTQC roadmap, resource estimation remains confined to the algorithmic layer. This restricts the adoption of QRE tools by researchers of other layers. Another consequence of the existing QRE design philosophy is that full-stack optimization across layers remains largely unsupported.

Existing QRE tools infer resource estimates using either the compilation or symbolic annotation method. Quantum software stacks, such as Qiskit, include sequential compilation passes [36, 37], including high-level synthesis, gate-set decomposition, layout selection, routing, scheduling, etc. Each pass transforms the circuit structure, thereby updating the estimates of various resource costs. QRE propagates through the stack and tracks the resources induced by the passes. Compilation provides accurate resource counts; however, it becomes computationally prohibitive for large design spaces. To mitigate this, QRE tools often use compilation at the algorithm layer to be flexible to user input, while the lower layers are based on symbolic models hardcoded for the FTQC stack. Symbolic annotation, though trivially scalable and interpretable, demands deep cross-domain expertise and is difficult to adapt as assumptions evolve. These gaps prompted us to redesign how QRE can be made flexible across regimes and stack, tractable at scale, yet requiring minimal domain expertise.

2.2 Quantum algorithmic profiling

Complexity theory characterizes algorithmic scaling in terms of input size, while deliberately abstracting away constant factors, lower-order terms, hardware effects, and system-level interactions. In practice, however, these effects often dominate real-world performance in resource-constrained environments. In classical computing, software profiling has long served as a practical complement to theoretical complexity analysis. Profilers such as gprof [38] empirically measure execution time, memory consumption, cache behavior, and call frequencies, enabling developers to identify performance bottlenecks that are typically out of scope in asymptotic analysis. This empirical perspective is particularly valuable when performance depends on implementation details, compiler decisions, or architectural features that are difficult to model analytically. Recent works [39, 40] have adapted this paradigm to quantum settings.

Complementary to profiling, static analysis techniques such as automatic amortized resource analysis (AARA) provide sound upper bounds on resource usage (typically runtime) through type systems and abstract interpretation. Recent work has explored hybrid approaches that augment static guarantees with data-driven Bayesian inference to improve robustness and tightness [41, 42]. While these methods demonstrate promising results for time and memory analysis in classical programs, their adoption in quantum computation [36, 43, 44] remains limited to verifiability. In particular, frameworks for jointly modeling multiple interacting resources, such as space, time, and fidelity, are still lacking.

Algorithmic profiling [45] extends classical software profiling by explicitly linking empirical measurements to inferred cost models. It acts as a bridge between computational complexity theory and runtime profiling, aiming to recover interpretable cost functions from observed execution data. Given a set of tuples mapping input characteristics to measured resource costs, an algorithmic profiler infers a functional relationship that approximates the underlying resource asymptote. The broader field of empirical algorithmics frames algorithmic profiling as the automatic inference of cost functions or asymptotic bounds from experimental data [46]. Superficially, this formulation appears to conflict with computability limits such as Rice’s theorem [47], which states that all non-trivial semantic properties of programs are undecidable. Resource usage is precisely such a property. However, this theoretical limitation does not preclude practical profiling for several reasons. First, profiling operates on finite executions and bounded input regimes, rather than on all possible program behaviors. Second, the goal is not exact characterization, but approximate prediction within a domain of interest. Third, profiling exploits regularities induced by algorithmic structure, compiler conventions, and hardware constraints, which significantly reduce the effective hypothesis space. Thus, algorithmic profiling adopts a pragmatic stance, aiming for actionable guidance rather than formal completeness.

Analogously, in the quantum circuit model, non-trivial resource properties, such as circuit depth after compilation, or logical error rates under noise, depend on the semantic behavior of quantum programs over transpiler transformations. Exact analytical characterization of such properties across arbitrary circuits and compilation stacks is therefore intractable and undecidable in general. However, as in classical empirical algorithmics, this limitation does not preclude practical estimation. AutoQuREO leverages quantum algorithmic profiling to empirically approximate these properties using sampled configurations and small-instance compilations and infer scalable cost models. Thus, quantum algorithmic profiling constitutes the data-generation stage of the surrogate modeling pipeline. The small-instance compilations produce the labeled tuples that map hyperparameter configurations to measured resources and serve as training data, and the empirical regularities these tuples expose justify modeling quantum circuit resources as learnable functions of circuit features. This explicit framing of QRE through empirical algorithmics implemented as a surrogate modeling pipeline, is novel in quantum computing.

Algorithmic profiling, however, is in itself a non-trivial modeling problem, trading off interpretability, generalization, and computational efficiency. For example, lookup-based models offer high fidelity but poor scalability, neural

networks improve predictive performance at the cost of transparency, while symbolic regression methods provide interpretable closed-form expressions but may struggle with high-dimensional parameter spaces. AutoQuREO addresses this trade-off by treating surrogate model selection and configuration as a meta-optimization problem, enabling systematic exploration of model families and hyperparameters. This ensures the quantum resource estimates remain scientifically informative across diverse QRE scenarios without the user manually refining the model. The model selection and configuration is further discussed in Sections 3.4-3.7.

3 The AutoQuREO framework

This section introduces the design and architecture of AutoQuREO, the proposed framework for automated, full-stack quantum resource estimation and optimization. We describe the core design principles underpinning the system, including flexible stack abstractions, modular interfaces, scalable surrogate modeling via algorithmic profiling, and an integrated optimization pipeline. The section further details the software design and architecture of our implementation.

3.1 Design principles and novelty

The accelerating pace of quantum computing research has shifted the focus from isolated innovation of hardware and algorithms to system-level solutions. Achieving a higher quantum technology readiness level (QTRL) [48] increasingly depends on the ability to rapidly perform DSE and deploy these design choices across the full QC stack. Currently, this process remains largely manual, fragmented across tools, and biased toward long-term FTQC assumptions. AutoQuREO is designed to address this gap by treating quantum resource estimation as an integral component of quantum software development. By automating the process and incorporating optimization as a post hoc step, we augment the QRE workflow beyond a static accounting exercise.

The framework is built around four core novelties $\mathcal{N}$, each addressing a fundamental limitation of existing QRE approaches.

- **$\mathcal{N}1$ - Full-stack flexibility:** AutoQuREO imposes no fixed notion of layers, architectures, or fault-tolerance assumptions. Instead, users define arbitrary stacks composed of modular layers. Layer interfaces are explicitly defined using the adapter design pattern, enabling compatibility across the stack and reuse across use cases. This allows our tool to be of relevance to quantum researchers across the stack, beyond quantum algorithms.
- **$\mathcal{N}2$ - Library for rapid prototyping:** The framework provides a library of composable code implementations (called modules) and models that allow users to instantiate end-to-end QRE pipelines with minimal boilerplate code, lowering the barrier to systematic co-design studies.
- **$\mathcal{N}3$ - Surrogate synthesis of resource models:** To overcome the scalability limits of compilation-based estimation and the domain-expertise requirement of symbolic annotation, AutoQuREO introduces surrogate synthesis for resource models. Models vary in estimation cost and accuracy, including neural networks, symbolic regression, and lookup-tables. Advanced AI methods, such as symbolic distillation of neuro-evolved networks, are used to autonomously synthesize and fine-tune scalable models.
- **$\mathcal{N}4$ - Automated optimization:** Resource estimation is embedded within an optimization loop that supports multi-objective user-guided Pareto front analysis and downstream deployment. This enables targeted analysis of performance bottlenecks. Eventually, the optimized configurations can be compiled and deployed on target backends. AutoQuREO turns the stack configuration from a passive object of analysis into an active decision variable, and integrates estimation, optimization, and deployment within a single interface, so that researchers need not switch to a separate quantum SDK for circuit generation and cloud execution. This tight coupling of QRE with deployment is atypical of existing QRE tools.

Together, these principles position AutoQuREO as a system-level digital twin for quantum computing stacks, designed to support improvement across NISQ, EFTQC, and FTQC regimes.

3.2 Layer definition

AutoQuREO represents a quantum computing stack as an ordered collection of layers, each encapsulating a functional stage of running an end-to-end quantum solution. Typical layers include input data preprocessing, quantum algorithms, quantum logic gate decomposition, quantum circuit routing, quantum error handling (e.g., error mitigation or error correction), quantum hardware execution, and output data postprocessing. Importantly,

the framework does not enforce a fixed taxonomy: layers may be added, removed, merged, or reordered within the constraints of architectural dependencies. The resource estimation is organized sequentially according to these layers.

Each layer can have multiple implementations, either as a module or a model.

- **Module:** is an implementation of a layer that can be compiled, and a quantum circuit is available as an output, which can further be used for QRE. For example, for a quantum logic gate decomposition layer, this can be implemented as Python code that takes in Qiskit circuits and decomposes them by transpiling them into an equivalent circuit composed only of quantum logic gates supported by the lower layers, say, using the quantum Shannon decomposition and Solovay-Kitaev decomposition. Resources estimated from modules are considered as ground-truths. However, it is important to note that transpiling circuits does not scale well for large sizes. Since we need to perform resource estimation across many DSE configurations, solely using modules becomes computationally intractable beyond toy use cases.
- **Model:** To circumvent the intractability of using modules for QRE, AutoQuREO introduces layer-specific resource models. Users can define their own model type, e.g., neural networks, symbolic regression, and lookup-tables. Models can be thought of as surrogates that take in the resources of the input to the current layer, a specific hyperparameter configuration for the current layer, and output an estimate of the updated resources expected upon adding the layer to the stack.

The general design philosophy is to start with modules for each layer, model them individually, and then perform DSE across all layers as models. Once the optimal configuration is analyzed, the modules can be tuned to it and deployed.

In the context of the framework, it is imperative to clarify four additional terms that we have used loosely so far:

- **Resources:** broadly refer to the set of metrics that influence a user's preference to prototype a quantum solution in a certain way. For example, it may be the number of qubits required to deploy a quantum algorithm for a specific application, which may or may not be available in the selected quantum hardware. Typically, we consider the number of qubits, the number and types of quantum logic gates, the quantum runtime, and the fidelity of the quantum circuit. The AutoQuREO framework allows users to seamlessly add other resources of interest (e.g., energy consumption or active volume) as long as a specification for measuring or estimating them for a candidate configuration is provided.
- **Hyperparameter:** Each layer (strictly need not, but typically) has some tunable hyperparameters. These can be tuned to holistically optimize a full-stack quantum prototype.
- **Hyperparameter bounds:** Each hyperparameter of a layer has some typical range (or list of options, without any particular ordering) within which the QRE would be conducted. These can be specified as a list, e.g., (A,B,C), or as a range and a step size, e.g., (1 to 100 in steps of 20), or as a continuous range bound, e.g., (0.900 to 0.999).
- **Configurations:** When there are two or more independent hyperparameters of a layer, the permutation of their choices leads to different configurations of the layer. For example, say Layer X has 2 hyperparameters with H_1 having options (1,2,3) and H_2 having options (A,B); then the $3 \times 2 = 6$ configurations of Layer X are (1A,1B,2A,2B,3A,3B). Configurations of subsequent layers are combinatorially expanded from those of previous layers.

3.3 Adapters

AutoQuREO implements a plug-and-play design for modules and models via the adapter design pattern [49]. Adapters for each module/model maintain a specific Python-based template to define hyperparameters and log the resources. The modules and models might even be third-party tools that use different coding styles or languages (such as t|ket) [50]). Multiple adapters can also be composed into a composite adapter, which is useful when their corresponding hyperparameters are coupled, so that only the valid joint configurations are explored.

Figure 2 presents an example use of the AutoQuREO API. After installing the Python software package, a user starts by importing the package and creating an object with an experiment name (lines 1-5). The next step is to define the adapters for each layer (lines 7-22). In this example, we define two adapters: (1) `module_adapter_name` a quantum-compilation type, and (2) `model_adapter_name` a symbolic-regression type. Note that these might already be available in the library, so the user can import them directly from the adapter library. The user might also define their own adapters for their own modules. The module (or model) might be present in the AutoQuREO

library (line 7) or accessed from a user-defined location. The adapter is identified by the function name; the input interface of a dictionary of hyperparameter configuration and any data from the previous layer; and the output interface of a dictionary of resources, data from this layer to the next, and the confidence estimation of this adapter (lines 10-11 and 17-18). The encapsulated module (or model) is accessed inside the adapter with the input parameters (lines 12 and 21). For modules, a quantum circuit is returned and then inspected for various resources (line 13), whereas for models, the resources are returned directly (line 21). Though we show only the number of qubits as an example, multiple such resources of interest might be added to the dictionary. The layer data for modules might include the compiled quantum circuit when required for downstream layers, whereas for models, this is not available. Finally, the confidence score is typically 1.0 for resources assessed from direct circuit inspection, whereas for models it is fetched from the model adapter from the library (line 18), and updated during training, as explained in Section 3.7.

Once the adapters are defined (or imported from the adapter library), they are added to a specific layer of the project, along with the adapter type metadata (lines 27-28). After all the adapters are added, the layer is built, with the parameters specifying the layer name, the hyperparameter names, and their corresponding ranges (lines 24-25 and 30). The layer building process includes (i) expanding the hyperparameters to configurations, (ii) choosing a particular adapter, and (iii) adding the resource estimates for each configuration to each existing configuration up to the previous layer to form the new layer of nodes in the resource tree. These steps will be explained in further detail in the following sections. The layer building progress can be tracked, as it might take significantly longer for large DSE experiments.

```

1  from autoqureo import AutoQuREO
2  from autoqureo.utils import import_model
3
4  experiment_name = 'scenario'
5  qre_object = AutoQuREO(experiment_name)
6
7  from library.modules import module_call
8  from library.adapters.free import model_adapter_call
9
10 def module_adapter_name(hyperparameter_configuration: dict, prev_layer_data: dict) -> tuple[dict, dict, float]:
11     resources, layer_data, confidence = dict(), dict(), 1.0
12     compiled_circuit = module_call(hyperparameter_configuration, prev_layer_data)
13     resources['num_qubits'] = compiled_circuit.num_qubits
14     layer_data['quantum_circuit'] = compiled_circuit
15     return resources, layer_data, confidence
16
17 def model_adapter_name(hyperparameter_configuration: dict, prev_layer_data: dict) -> tuple[dict, dict, float]:
18     resources, layer_data, confidence = dict(), dict(), model_adapter_call.CONFIDENCE
19     model_name = 'model_1'
20     model_call = import_model(model_name)
21     resources = model_call(hyperparameter_configuration, prev_layer_data)
22     return resources, layer_data, confidence
23
24 layer = 'layer_name'
25 hyperparameter_range = [1,2,3]
26
27 qre_object.adapt_layer(layer_name = layer, adapter_type = 'quantum-compilation', adapter = module_adapter_name)
28 qre_object.adapt_layer(layer_name = layer, adapter_type = 'symbolic-regression', adapter = model_adapter_name)
29
30 qre_object.build_layer(layer_name = layer, hyperparameter_name = hyperparameter_range)

```

Figure 2 API template for AutoQuREO project creation, adapter definition, and layer building.

Adapter type is metadata used to select a specific adapter for performing the QRE on a layer. Typically, it is one of the following types: quantum-compilation, lookup-table, neural-network, symbolic-regression, preprocessing, postprocessing, or executor. In addition to the type, each adapter has two additional metadata fields. Cost refers to the classical computational time to estimate the quantum resources. For example, as discussed earlier, estimating the resource via quantum compilation (or module) is not scalable and thus incurs a high cost. In contrast, a trained neural network at inference can quickly estimate resources. Confidence refers to a measure of the closeness of the resource estimate to the actual resource, reflecting uncertainty due to limited data or extrapolation. Obviously, since compilation is the ground-truth, the confidence of modules is 100% or 1.0. For models, the confidence is typically less than 1.0. The significance of these metadata and the process of estimating and updating these will be clarified in Section 3.5 and 3.6, respectively.

Figure 3 presents the internal workflow of AutoQuREO, to aid the comparison with existing QRE tools. The layers are not predefined in AutoQuREO. Users can flexibly add or remove layers based on their QC stack of interest. The specific layers shown are just an example. Standard layer definitions are available in the library for rapid prototyping of full-stack use cases. In existing tools, the quantum algorithm layer is typically the only user-defined component, leaving the lower stack monolithic. Each layer has its corresponding set of tunable hyperparameters used for DSE and eventual optimization by AutoQuREO. Since no layer is treated differently, these hyperparameters can thus be used to optimize any/all chosen layer/s, enabling co-design. Each layer is defined by either a code implementation that can be compiled into a quantum circuit or by one or more surrogate models that map hyperparameter configurations to resources. Autonomously synthesizing these models forms the core novelty of AutoQuREO. The code and model across layers follow a standard adapter interface, allowing easy plug-and-play with the tool. During the build process, one of the adapters is selected to estimate the resources corresponding to the configurations. These resources are cumulated across the layers to produce the full-stack resource estimate. This estimate is valuable for bottleneck and sensitivity analysis. AutoQuREO also allows Pareto optimality analysis to select an optimal configuration for deployment on the actual quantum computing platform.

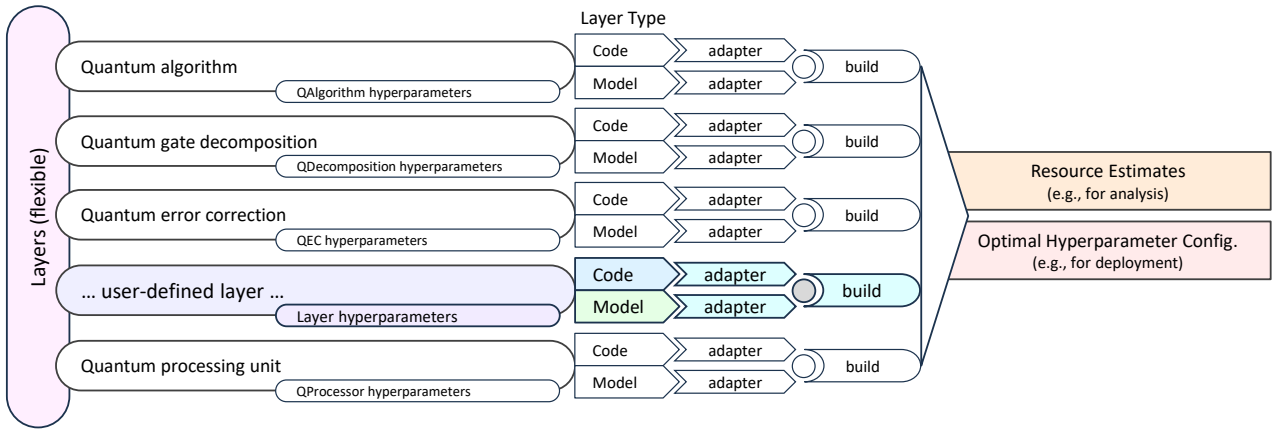


Figure 3 Internal workflow of AutoQuREO. A quantum computing stack is represented as a sequence of user-defined layers, each associated with hyperparameters and one or more adapters. For a given configuration, each layer produces resource estimates that are cumulatively aggregated across the stack. Surrogate models replace compilation where applicable, enabling scalable estimation. The aggregated resources are used for downstream multi-objective optimization and hardware deployment.

3.4 Resource models

To achieve scalability, AutoQuREO employs various surrogate modeling [51] techniques for resource estimation. Some archetypal ones are discussed herein. Lookup-table [52] is employed in regimes where precomputed values can effectively be substituted at inference-time to accelerate the QRE without forgoing accuracy. Symbolic regression can be used when interpretability and extrapolation are critical. Various library models use PySR[53] to perform symbolic regression on a dataset using genetic programming for the equational structure and simulated annealing for the numerical regression. Neuro-evolution using augmented topologies (NEAT) [54] is another technique employed in some models, especially as a flexible approximator for highly non-linear resource behavior. As a neuro-symbolic (NeSy) technique, these two can be combined by using PySR for symbolic distillation on a trained NEAT model. This denotes a two-stage procedure. A NEAT network is first evolved to fit the profiled resource data as a flexible sub-symbolic approximator, providing accuracy and robustness over irregular data. PySR then distills this trained network into a compact closed-form expression. The composite estimator combines the predictive quality of the network with the transparency and low inference cost of a formula. NEAT captures intricate empirical patterns, while PySR extracts structured representations that are formalizable and compositional. NeSy aligns with the so-called third wave of artificial intelligence (AI) that seeks to combine sub-symbolic efficiency with symbolic interpretability. To the best of our knowledge, AutoQuREO is the first to systematically apply NeSy-AI to quantum computation in general and resource estimation in particular. NeSy-QRE enables the framework to reconcile competing demands of scalability across large design spaces, robustness to architectural variation, and transparency necessary for scientific insight and trust.

3.5 Adapter metadata and arbitration

An adapter includes various metadata to guide the QRE process. When a model is encapsulated by an adapter, an initial estimate of the cost and confidence metadata is assigned based on the model type. This estimate is based on model training conducted prior to the QRE process, known as warm-start. The cost is based on the model's inference time on the test set, while the confidence reflects the test-set performance (e.g., via RMS error or F1 score).

During the QRE process, an agent arbitrates among available module and model adapters for each layer. Agents can be configured with a persona/policy as a scoring function, which encodes preferences such as accuracy versus speed, interpretability, and extrapolation compatibility for generalization. For example, an agent may prefer a quick-and-dirty estimate for fast QRE, while another agent may focus on getting a better estimate, trading off longer time to reach the optimal prototype. We use the reinforcement learning terminology of agent and policy to bespeak the incorporation of more advanced arbitration in the future. The adapter with the highest score is selected to estimate the QRE for the layer. When an adapter is chosen by the agent, based on the actual time the QRE for the particular configuration took, the cost metadata is updated. The confidence parameter is updated only during model retraining via lifelong learning, as explained in the following section.

3.6 Surrogate synthesis

Resource models, as introduced in the previous section, alleviate the scalability issue with QRE while being amenable to non-experts in quantum complexity analysis. However, generating the training data, training the models, and evaluating them is fairly involved, often requiring multiple iterations and prior working knowledge of AI. As illustrated in Figures 4a and 4b, surrogate synthesis automates and optimizes the entire QRE workflow with minimal user intervention.

The surrogate synthesis process proceeds in several stages, as shown in Figure 4c. To initiate the process, the user can inspect available adapters of a layer and their corresponding cost and confidence metadata. Based on the use case, the total QRE time for an adapter can be estimated and cumulated across the full stack. If this total time is prohibitive, the user can decide to synthesize a new surrogate model to accelerate the QRE DSE. The bottleneck adapter is chosen as the input adapter, and the user specifies a model class for the surrogate model. Each adapter also stores additional metadata of the typical hyperparameter ranges it is expected to work with. For modules, this corresponds to sizes that can be compiled tractably, while for models, it reflects the input data on which they have been trained. This metadata of the input adapter is accessed to generate the hyperparameter configurations. Then, the input adapter is invoked with these configurations to generate training and test data for the surrogate. Note that in this process, the input adapter need not be a module type; for example, a symbolic model can be derived from a neural network model, or vice versa. The surrogate model is then trained using specialized routines for each type. It is then evaluated on the test data to update the cost and confidence metadata. If these metrics are not within the user-set acceptable bounds, the model's hyperparameters are further tuned via Optuna [55], then retrained and reevaluated (for a set maximum number of trials). The final surrogate model is saved in the library, and the corresponding adapter code is synthesized, maintaining the same interface as the input adapter. This output adapter is also saved in the library for seamless addition in the QRE use cases.

3.7 Lifelong learning

AutoQuREO supports a lifelong learning mechanism [56] in which resource estimation data generated for each evaluated configuration are persistently retained and reused to improve surrogate models across the stack. When an adapter of module type is selected by the agent, it is treated as a source of ground-truth supervision. This is propagated to retrain other models of the layer if the corresponding adapter metadata indicates that lifelong learning is supported and enabled for them. The internal process of lifelong learning is similar to surrogate synthesis, and also updates the cost and confidence metadata.

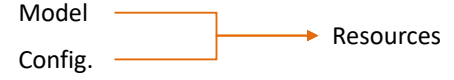
This continual model update enables AutoQuREO to progressively replace expensive compilation-based estimation with efficient approximations. In a typical use case, if we start with a module and a model corresponding to a layer, after reaching a threshold for arbitrating the module for the QRE during lifelong learning and improving the model, the model's metadata would be favorable to the agent's scoring function. Thereafter, not only for subsequent configuration's QRE, but also for further use of the layer (in the same project or another use case), the model can serve as a trusted and efficient resource estimator.

Existing QRE method (compiler wrapper)



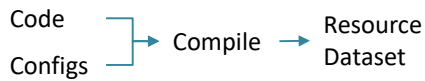
Problem: Compilation is slow and only possible for small sizes and few configurations.

Existing QRE method (symbolic annotation)



Problem: User must be a QRE expert to specify the resource model. QRE effectively is a trivial calculation.

AutoQuREO (resource modeling)



Solution: Compilation data for small configs are used to create a resource model.

AutoQuREO (resource estimation)



Solution: Automated modeling allows users to effortlessly use QRE insights for large configurations.

Problem: Training data generation and fine-tuning of resource model is manual and non-trivial.

Solution: Autonomous surrogate synthesis of quantum resource models

(a)

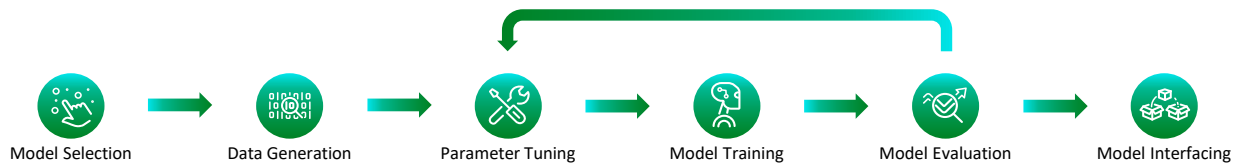
Features	Resource Estimation Methods			
	Symbolic	Compilation	Modeling	Surrogate Synthesis
Domain knowledge required	High	Low	Low	Low
Programming/ML knowledge required	Low	High	High	Low
Compute power required and run-time	Low	High	Mid*	Mid*
Scalability to large problems	High	Low	High	High

* Note:

Compute cost of Modeling is slightly less than Surrogate Synthesis, though the Data Generation part is identical.

In Surrogate Synthesis, the entire model training and evaluation also runs in a loop for model hyperparameter optimization, whereas manual Modeling can be done in a few trials (provided user has AI/ML expertise.)

(b)



(c)

Figure 4 Surrogate synthesis workflow and comparison with existing QRE methods. **(a)** Comparison of QRE workflow of compilation-based, symbolic-annotation-based and the proposed modeling-based approach, highlighting trade-offs in scalability and automation. **(b)** Comparison of resource estimation features across criteria such as required expertise, computational cost, and scalability. **(c)** Surrogate synthesis pipeline including model selection, data generation, model tuning, training and evaluation, and adapter sketching. This enables AutoQuREO to perform scalable resource estimation.

3.8 Resource analysis and optimization

The QRE iteratively estimates the cumulative resources for each layer based on the modules/models. This is referred to as building a layer for the project, as explained previously in Section 3.3. AutoQuREO's build process internally updates a tree data structure called the resource tree. Each cumulative design space up to a layer of the stack corresponds to a level of the tree. Each edge in the resource tree corresponds to a specific hyperparameter configuration for a particular layer. Each node of the resource tree stores the hyperparameter configurations up to that node from the root, along with the cumulative resources at the end of processing for that layer. For example, if layer 1 has 10 configurations and layer 2 has 6 configurations, the first layer of the tree will have 10 nodes, while the second layer will have $6 * 10$ nodes, for each possible configuration of the entire compute stack. Thus, each progressive layer represents the cumulative configuration of all the layers up to that point. An example tree is shown in Figure 5 for 4 layers. The layer names are annotated with colors on the left. The optimal leaf nodes after the optimization process, as discussed shortly, are highlighted with a black border. Each node can be inspected to view the corresponding resources and hyperparameter configurations. The HTML version of the tree can be interactively explored for the node/edge data via a pointing device. However, visualizing the tree becomes unruly for realistic DSE due to the combinatorial explosion. To keep the exploration tractable, AutoQuREO samples configurations from the specified hyperparameter ranges under user control rather than exhaustively enumerating every combination. The resources at each node are estimated by surrogate models rather than by compiling every configuration, so the design space is traversed over inexpensive model evaluations. Besides a configurable sampling control, more advanced search strategies, such as grid search or differential evolution, would be considered in future releases.

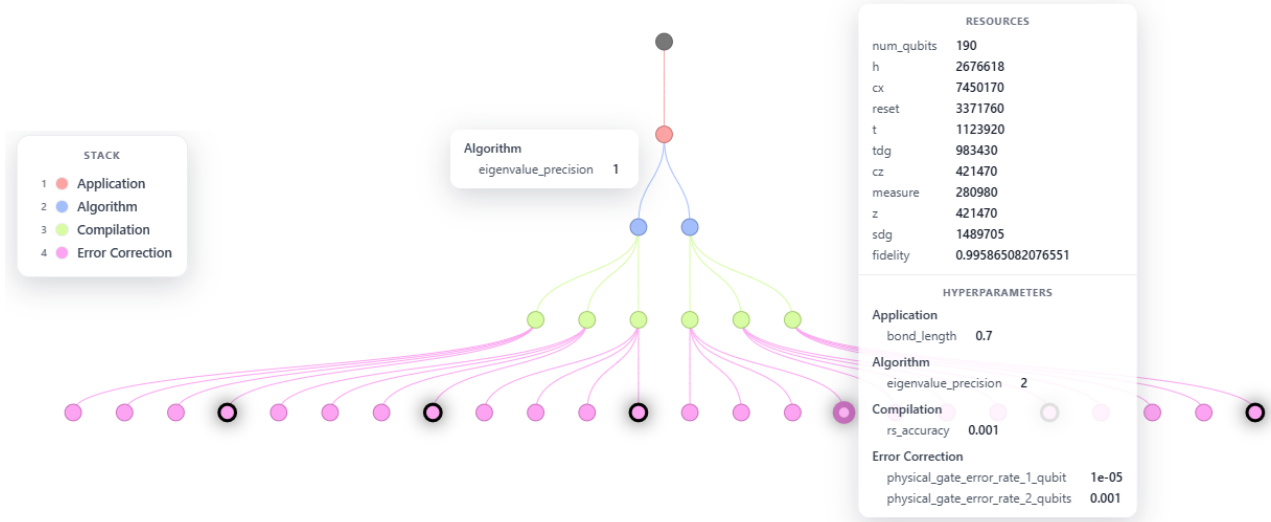


Figure 5 An example resource tree representation of the design space. Each level corresponds to a stack layer, and edges represent hyperparameter choices. Nodes encode cumulative configurations and aggregated resource estimates. Leaf nodes correspond to complete system configurations, among which Pareto-optimal solutions (highlighted nodes) are identified based on user-defined objectives.

To further analyze the configurations, AutoQuREO allows plotting any 2 or 3 quantities of interest from the full set of hyperparameters and resources of all layers. This can be plotted for all leaf nodes or for a subset. For example, the space (number of qubits) vs time (circuit depth) vs error (observable fidelity) plot for the leaf nodes.

Resources and hyperparameters jointly define a multi-objective optimization problem. We emphasize that this optimization is distinct from the surrogate modeling step. Models first estimate the resources across the design space. The optimization then operates on these estimates by filtering Pareto-optimal configurations. Because the estimates come from models rather than full compilation, the design space is searched without compiling every configuration, and only the finally selected configuration is compiled for deployment. The optimization is guided by the user's goal in the use case. These define the actions for each resource or hyperparameter. Actions can be set to either minimize or maximize, as shown in Figure 6. These actions guide the optimization, while the other resources and hyperparameters (e.g., layer 1 hyperparameter 1 and resource 1 are unspecified in the example) are outputs that the user wants to study. Note that the min/max actions might not be sufficient to single out a configuration. There might be trade-offs that form a Pareto frontier. For example, if the goal is to minimize the

number of qubits while maximizing the input problem size, the Pareto front typically filters out configurations that do not dominate at least one of the goals, keeping the other fixed.

```

1 actions_hyperparameters = {
2     layer_1: {layer_1_hyp_1: 'min',
3               layer_1_hyp_2: 'max', },
4     layer_2: {layer_2_hyp_2: 'max'}
5 }
6
7 actions_resources = {
8     resource_2: 'min'
9 }
10
11 pareto_configs = qre_object.optimize(actions_hyperparameters, actions_resources)
12
13 circuit_to_deploy = qre_object.get_circuit(random.choice(pareto_configs))

```

Figure 6 API template for AutoQuREO resource optimization and deployment. Users define optimization objectives over hyperparameters and resource metrics. The framework performs multi-objective optimization to identify Pareto-optimal configurations, which can then be compiled into executable circuits. Estimation, optimization, and deployment are integrated within a unified interface.

Once optimal configurations are identified, the layers can be compiled and deployed on target backends, closing the loop between design and execution. The framework is designed for integration with existing platforms, including support for Fujitsu’s quantum SDK [57] and backend, enabling seamless transition from resource estimation to hardware execution.

3.9 Workflow of a QRE scenario

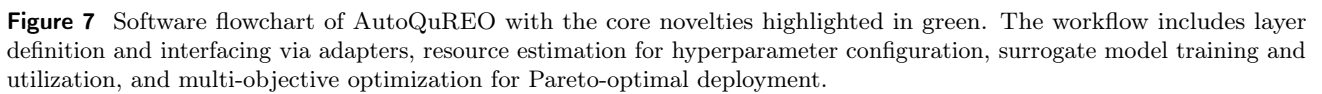
To effectively use AutoQuREO, a user would typically start by defining a blueprint of the layers to add to the QC stack and the trade-offs they are interested in analyzing and optimizing. We refer to such a non-trivial use case of the AutoQuREO tool as a scenario.

After installing AutoQuREO, users may interact with the framework through either a Python-based application programming interface (API) or a graphical user interface (GUI). The scenario is used to add the corresponding layers to the QRE stack, either directly from the library if available or manually by the user. The QRE is performed iteratively per added layer, and the user can track the progress (especially for large-scale DSE). The final estimates can be visualized and analyzed with custom tools within AutoQuREO. These aid the user in analyzing the design bottlenecks and addressing them in the layer definitions. Once the user is satisfied with the optimization, the optimal configuration might (i) be the end-goal for exploratory research on projected configurations and resources (e.g., how many qubits are needed to run a problem size of interest), or, (ii) be deployed on an available hardware backend (e.g., what is the largest problem size that can be run on a hardware of interest). Note that, though the surrogate synthesis of resource models is the core novelty, the user is effectively agnostic to the details of this involved process.

The API enables scripting of large-scale design space exploration and integration into existing SDK workflows, while the GUI supports interactive inspection of resource trade-offs, Pareto fronts, and profiling results. Internally, execution is orchestrated by a controller that manages data flow between layers, invokes appropriate estimators, and records provenance for reproducibility.

3.10 Software architecture and execution flow

The detailed software design of AutoQuREO is depicted via the flowchart in Figure 7. Each block **B** is numbered for ease of following the discussion. Figures 8a, 8b, and 8c are insets further detailing the blocks **B13**, **B15**, and **B19** respectively.



- We will consider a scenario with 2 layers, L_1 , followed by L_2 .
- Let's assume L_1 has 2 tunable hyperparameters with the following values: $L_1H_1 := (A,B,C)$ and $L_1H_2 := (D,E)$. To estimate the resource, let's assume L_1 has 3 choices of adapters:
 - L_1A_1 of type quantum-compilation (available in the module library)
 - L_1A_2 of type lookup-table (new empty model user will create and use), and
 - L_1A_3 of type symbolic-regression (new model user will create, warm-start, and use).
- L_2 has 1 tunable hyperparameter $L_2H_1 := (F,G)$. L_2 has only 1 adapter.
 - L_2A_1 , of type neural-network (imported from a 3rd-party library).
- Let's also assume that we are interested in 3 quantum resources at each stage, namely (R_1,R_2,R_3) .
- Our interest is to specifically study the dependence of R_2 and L_1H_2 , and what are the associated optimal values of L_1H_1 , L_2H_1 , R_1 , and R_3 for minimizing R_2 while maximizing L_1H_2 .

The AutoQuREO session starts in **B1** by creating a new project with a user-specified project name. This project can be saved and loaded later. The saved/loaded data includes layer information (name, hyperparameters, adapters), agents, the resource tree, and optimization actions with the corresponding optimized configurations. For the rest of this discussion, let's consider an empty project.

In **B2**, the user decides if the current project layers match the layers in the scenario blueprint. Currently, no/false, as we want to add the 2 layers, starting from an empty project. There's no point in jumping to **B19** without adding any layer to optimize over, and it would result in an error flag. Let's start with adding layer L_1 .

Adapter definition and library

The next step is to add the layer adapters for this layer. As in the blueprint, we need to add 3 adapters; thus, the **B3-B4** loop would be executed 3 times. For the 1st time, i.e., for L_1A_1 , the user selects a quantum-compilation module from the module library **B5**. The adapter code (i.e., the Python code that interfaces with the library function) is also available in the library **B5**, so the user can use it directly. After successful interfacing, adapter L_1A_1 gets linked to L_1 . This adapter-layer map is logged in the database **B8**.

The flow returns to **B4** via **B3** now for the 2nd adapter L_1A_2 . This is a lookup-table type estimator that the user wants in this scenario, so that whenever the same configuration needs to be estimated, it is faster to retrieve the saved results than to recompile it with L_1A_1 . Since this is a new model, the user defines and adds it to the AutoQuREO model library **B6**. The adapter L_1A_2 for the lookup-table is then added via **B4** to **B8** for the layer L_1 .

We move on to the last adapter of this layer, L_1A_3 . This time, say, the user wants to compress the resource estimate into a set of equations that map the hyperparameters to the resources. The user creates the new symbolic model, say, as a set of SymPy [58] expressions, and defines a way to update/train the model, say, via PySR [53] or similar tools [59, 60]. To initialize the model at this stage (i.e., before being used in the QRE process), the user might warm-start the model by training it on the compilation module of L_1A_1 for an input dataset of the user's choice (e.g., a set of Haar random quantum states/unitaries, or a set of benchmark quantum algorithms like MQT Bench [61] or QARP [62]). Note, this warm-start training can also be done independently before running the AutoQuREO session, thus keeping the warm-started model already available in the model library while adding the corresponding adapter. Warm-start learning works similarly to lifelong learning, as described later. In either case, the adapter L_1A_3 of the symbolic model is listed in **B8** against L_1 . From **B4**, we return back to **B3**. This time, we have added all three adapters as per the Scenario blueprint. Next, we will estimate the resources for the L_1 layer.

Agent-based adapter arbitration

In **B9**, we begin preparing for this step by initializing an agent that arbitrates between the 3 adapters in this layer. There are a few predefined agent policies/personas available within AutoQuREO, but users can create and fine-tune their own agents. The agent assigns initial cost and confidence estimates for the 3 adapters. Say, in our case, the cost is (0.9, 0.4, 0.3), while confidence is (0.9, 0, 0.2). The default agent's policy is simple: pick the adapter with the highest confidence-divided-by-cost score, i.e., (1, 0, 2/3) for this case. Thus, initially, the agent would pick L_1A_1 to estimate the resource for this layer. Note that the values the agents assigned reflect the general understanding that the compilation is costly but close to the true value. The user's confidence in the symbolic equations can be reflected in the initialization by manually overriding the agent's default assignment. The lookup-table is initially empty, thus its confidence is zero.

Configuration expansion

After that, we move to **B11**. In this step, we expand the hyperparameters to configurations. The hyperparameter bounds of $L_1H_1 := (A, B, C)$ and $L_1H_2 := (D, E)$ are invoked from the user via **B10**. These are combined to create the configurations for layer L_1 as (AD, AE, BD, BE, CD, CE).

Resource estimation

The loop from **B12** to **B16** estimates and logs the resources for each configuration. For each configuration, the agent arbitrates one of the adapters to generate a resource estimate. This operation of **B13** is further expanded in inset A, in Figure 8a. As discussed before, in **B13.1**, the agent uses its policy to score the available adapters of the layer from **B8** based on the current cost and confidence. The highest-scoring adapter is selected in **B13.2** to estimate the resource. The score is based on the agent's policy function. Then, in **B13.3**, this chosen adapter is fed with the configuration, and the resource estimate is obtained.

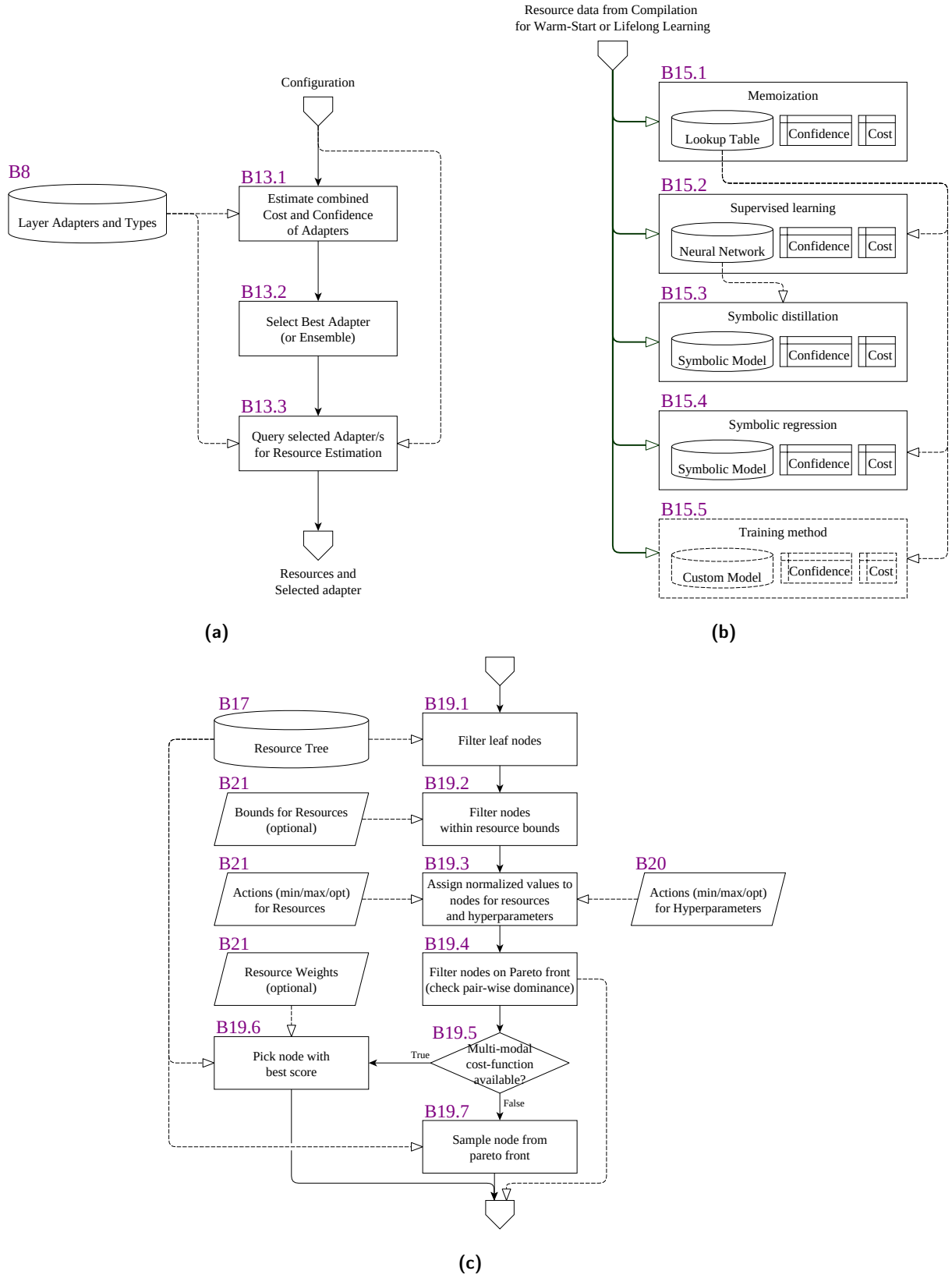


Figure 8 Detailed views of key blocks in the flowchart. **(a)** Inset A for block B13 for adapter arbitration based on cost and confidence metrics. **(b)** Inset B for block B15. The model creation and update follow the surrogate synthesis process described in Figure 4c incorporating warm-start and lifelong learning. **(c)** Inset C for block B19 for multi-objective Pareto filtering.

Lifelong learning of surrogate models

Back in the main flowchart of Figure 7, we move to B14, where we check whether the adapter the agent chose is a compilation type or a model. In the event of the layer not having any compilation type adapter, or if it had not been chosen by the agent, we skip to block B16, which stores the configuration and the corresponding resource estimate from the adapter into the resource tree in B17.

Alternatively, if the chosen adapter is of quantum-compilation type, we move to B15 to update any model adapters that allow lifelong learning. This is further explained in inset B, in Figure 8b. In contrast to traditional machine learning, lifelong learning refers to an AI system's ability to continuously learn and adapt to new data and tasks throughout its deployment, without needing to be retrained from scratch. AutoQuREO implements lifelong learning based on compilation-based resource estimates for 4 typical models: memoization via lookup-table in B15.1, supervised training of neural network in B15.2, symbolic distillation of neural network B15.3, and symbolic regression of lookup-table in B15.4. Each of these includes a specification of how the corresponding model is stored, and how it can be retrained, updating both the model and the confidence. Note that users can build their custom models and lifelong learning routine in B15.5. Examples of such models include the Kolmogorov-Arnold network, transformer neural networks, diffusion neural networks, neuro-evolution, etc. The model update process follows the surrogate synthesis steps described in Section 3.6 via Figure 4c. When a model is updated, it does not affect the resource estimates of the current configuration. The updated, higher-confidence model metrics affect the QRE of subsequent configurations; for example, the agent might now choose a different adapter.

Resource tree and layer composition

Back from the inset into the main flowchart of Figure 7, once the configuration is added in the resource tree in B17 via B16, the flow returns to B12 for the next configuration. Similarly, all configurations and their corresponding resources are stored in the resource tree. When all the configurations have been explored, the flow returns to B2. The resource estimation of Layer L_1 is complete at this stage. The resource tree can be visualized in B18.

Following our scenario, we move on to layer L_2 . There's a single adapter L_2A_1 of type neural-network. The flow returns to B4 via B3. This time, the user wants to use a trained neural network from a 3rd party, available as a TensorFlow model in B5 (alternatively, it can also be a cloud-based AI-powered transpilation [63] service). The user now has to write the adapter code corresponding to L_2A_1 , that loads the neural network (or connects to the cloud via an access token-based API) and returns the resource estimates from the model. There's no scope to update this model on the fly. The adapter is enlisted again in B8. The flow returns to B3, and since there are no more adapters, we move to B9 where the agent initializes the adapter with the default/expected cost and confidence of the neural network (which, of course, the user can override). The single hyperparameter L_2H_1 maps to the configurations (F,G) of the layer in B11. However, note that this is the 2nd layer; thus, each configuration must be applied to all configurations already considered in the previous layer (6 in our case, for the 1 layer before it). Thus, the full configuration becomes (ADF, ADG, AEF, AEG, BDF, BDG, BEF, BEG, CDF, CDG, CEF, CEG). For each of these 12 configurations, the loop from B12 to B16 estimates and logs the resource in the tree, building the next layer. Since there is only a single adapter, the agent always queries L_2A_1 in B13. Since there are no compilation-type adapters, the neural network cannot improve through lifelong learning, even if the 3rd party has that feature. After the layer has been built, the flow returns to B2. We can again visualize the updated resource tree of B17 at this stage via B18. At this stage, our *resource estimation* flow is complete.

Multi-objective Pareto-optimization

Next, we discuss the *resource optimization* feature of AutoQuREO. The optimization requires actions for each hyperparameter and resources for the current stack. Recall our optimization goal from the scenario blueprint. The action for the Hyperparameters in B20 would specify (L_1H_1 :opt, L_1H_2 :max) for L_1 and (L_2H_1 :opt) for L_2 , while for the Resources in B21, it would be (R_1 :opt, R_2 :min, R_3 :opt). B19 optimizes as further explained in inset C, in Figure 8c. The leaf nodes in the resource tree of B17 are filtered out in B19.1, as they correspond to a cumulative set of configurations and resources across all layers. These nodes can optionally be filtered by a hard-bound on resources in B19.2, specified by the user via B21. This is required as during the hyperparameter exploration, the user might not have a clear idea of how much of each resource the final prototype would need. Many of these configurations might require resources beyond our current interest (e.g., a larger quantum computer than we currently have). Filtering out such unreasonable settings helps declutter downstream resource analysis and optimization. Thereafter, based on the supplied actions of hyperparameters in B20 and resources in B21, the hyperparameters/resources with only min or max actions are filtered out in B19.3. Then, they are normalized such that the minimization action is inverted to maximization. Recall, in our scenario, the optimization target of

$(R_2:\text{min}, L_1H_2:\text{max})$ becomes $(1/R_2:\text{max}, L_1H_2:\text{max})$. Thus, in B19.4, we can jointly maximize over the selected hyperparameters/resources. As described in the glossary, these can be multiple configurations that are Pareto-optimal for a set of actions. The user can optionally supply an additional cost-function to weigh these actions in B21. If that is the case, from B19.5 we choose the node with the best score, B19.6. Alternatively, a sample node from the Pareto front is returned in B19.7. Note that the full node information is returned, not just the optimal configuration based on the actions. This is accessed from the resource tree in B17, which contains information on all the associated hyperparameters and resources for the chosen leaf node. The full list of Pareto nodes for the optimization scenario is also returned by B19.4 for analysis. Back in the main flowchart of Figure 7, this optimal node is returned to the user in B22 as the output of the full-stack optimization. The user can also analyze the Pareto front in B26 and plot subsets of the hyperparameters/resources as 2D/3D plots specified via B27.

Compilation and deployment pipeline

Once the optimal hyperparameter configuration that adheres to resource constraints and the scenario goal is obtained, we can compile the actual code for this configuration in B23. Note that if all the adapters used for the node expansion were of the quantum-compilation type, this full implementation is already available in the resource tree and only needs to be retrieved from B17. However, this is not the advocated approach, as fully compiling so many configurations to arrive at the optimal setting quickly becomes intractable. Thus, we would likely have models that estimated resource requirements for at least some layers. If we have used a model in any layer, we now have to compile the actual code using a quantum-compilation adapter for that layer, provided it is available (otherwise, we inform the user that a way to compile the code is missing). The compiled quantum circuit for the optimal configuration is then output to the user in B24 and can be deployed on a quantum computer in B25. Blocks B23 to B25 embeds the AutoQuREO tool in a broader quantum computing platform-as-a-service.

In summary, AutoQuREO introduces a flexible, scalable, and automated framework for full-stack quantum resource estimation and optimization. By combining modular stack abstractions, neuro-symbolic surrogate modeling, and integrated optimization workflows, the framework addresses key limitations of existing QRE tools. The following section demonstrates the practical impact of these design choices through a series of representative co-design case studies.

4 Exemplary co-design use cases

In this section, we demonstrate how AutoQuREO can be used on representative full-stack co-design problems in quantum algorithms, quantum compilation, quantum error correction, and quantum machine learning. For each use case, we first describe the problem motivation and related work, then define the scenario for resource estimation and optimization, including the stack layers, adapted modules and models, hyperparameters and ranges, and optimization actions. Thereafter, we present the results obtained from our framework and interpret the non-trivial insights obtained from the experiment. These examples are not intended as exhaustive benchmarks but rather as illustrative scenarios that highlight how full-stack, model-driven QRE can be performed systematically and scalably to gain novel, decisive insights. These demonstrate co-design challenges across NISQ, EFTQC, and FTQC regimes, illustrating how the layer abstractions, surrogate modeling workflows, and optimization primitives introduced in Section 3 enable exploration of otherwise computationally intractable design spaces.

4.1 Scenario I: Trotterized Hamiltonian simulation and error mitigation

Summary

- **Novelties used:** Full-stack flexibility ($\mathcal{N}1$), library support ($\mathcal{N}2$), and automated optimization ($\mathcal{N}4$).
- **Co-design variables:** Trotterization order, Trotter step count, noise-scaling factors, noisy backend, and zero-noise extrapolation (ZNE).
- **Key insight:** Joint exploration of Trotterization and ZNE identifies regimes in which error mitigation provides benefit.

Background

Hamiltonian simulation is a foundational quantum primitive for a wide range of applications in quantum physics and chemistry, enabling the time evolution of many-body systems that are intractable for classical methods. It is useful for computing dynamical properties, ground-state energies, and correlation functions, which are

essential for applications in materials discovery, molecular design, and condensed-matter analysis. Efficient and accurate Hamiltonian simulation [64] is widely regarded as one of the most promising pathways toward achieving practical quantum advantage, motivating this case study. In this study, we focus on Trotterized simulation of the transverse-field Ising model (TFIM), using magnetization as the observable of interest.

Digital Hamiltonian simulation approximates the time-evolution unitary,

$$U(t) = e^{-iHt}, \quad (1)$$

where $\hbar = 1$, using a sequence of gates implementable on a quantum computer. A general decomposition of the time-evolution unitary cannot exploit the inherent structure of the Hamiltonian. In most cases simulating physical system dynamics, the Hamiltonian can be written as,

$$H = \sum_{\ell=1}^L H_{\ell}, \quad (2)$$

where each term H_{ℓ} represents a basic interaction (such as a local coupling or a Pauli string) whose short-time evolution $e^{-iH_{\ell}\Delta t}$ can be efficiently compiled. Hence, the total evolution time t is split into r equal short-time slices of duration $\Delta t = \frac{t}{r}$. A circuit implementing the product formula $S_p(\Delta t)$ is then constructed to approximate the short-time evolution $e^{-iH\Delta t}$. Here, $S_p(\Delta t)$ denotes a p -th order product formula approximation to $e^{-iH\Delta t}$ [65]. Repeating this circuit r times yields an approximation to the full evolution as,

$$U(t) = e^{-iHt} = (e^{-iH\Delta t})^r = (e^{-i\sum_{\ell} H_{\ell}\Delta t})^r \approx (S_p(\Delta t))^r. \quad (3)$$

This process is known as Trotterization, and in practice, it concerns two interdependent choices:

- **Trotter steps (r):** determine how many slices are used. Each slice of duration Δt , together with a single application of the short-time circuit, is called a Trotter step. Here, r denotes the number of steps, and Δt denotes the step size. Increasing the number of steps, i.e., smaller Δt , reduces the approximation error, but increases the circuit depth because the short-time circuit must be repeated more times.
- **Trotter order (p):** determines how the single-step product formula $S_p(\Delta t)$ is constructed from the Hamiltonian terms $\{H_{\ell}\}$. The parameter p denotes the order of the product formula and characterizes how rapidly the approximation improves as $\Delta t \rightarrow 0$. For the single-step approximation,

$$\|e^{-iH\Delta t} - S_p(\Delta t)\| = \mathcal{O}(\Delta t^{p+1}). \quad (4)$$

For example, first-order Trotterization is expressed as,

$$S_1(\Delta t) = \prod_{\ell=1}^L e^{-iH_{\ell}\Delta t}, \quad (5)$$

whose single-step error scales as $\mathcal{O}(\Delta t^2)$ in Big- $\mathcal{O}$ complexity.

A higher-order means each step is a more accurate approximation to $e^{-iH\Delta t}$, so one can often use fewer steps (larger Δt) to reach a fixed target error, but at the cost of a more complicated circuit for each step, and hence more circuit depth.

The global error of this algorithm typically scales as,

$$\|U(t) - (S_p(\Delta t))^r\| = \mathcal{O}(r \Delta t^{p+1}) = \mathcal{O}\left(\frac{t^{p+1}}{r^p}\right), \quad (6)$$

up to problem-dependent constants determined by the Hamiltonian terms $\{H_{\ell}\}$. Hence, when running this algorithm on noisy quantum devices, a trade-off is observed. Using a higher-order product formula or increasing the number of Trotter steps reduces the algorithmic error, but typically increases the circuit depth. The deeper circuit is susceptible to noise, thereby increasing the physical error. Let $\langle O \rangle$ denote the expectation value of the observable of interest. The algorithmic and physical errors contribute to the overall error in the estimated observable, which can be bounded as,

$$\Delta\langle O \rangle_{\text{total}} \lesssim \Delta\langle O \rangle_{\text{alg}} + \Delta\langle O \rangle_{\text{phys}}. \quad (7)$$

This physical error can be reduced in postprocessing using quantum error mitigation (QEM) techniques, such as zero-noise extrapolation (ZNE) [66]. ZNE estimates the zero-noise expectation value by executing the circuit at multiple noise levels and extrapolating the results to the zero-noise limit. A common approach to generate noise-scaled circuits is unitary folding, which maps $U \rightarrow U(U^\dagger U)^n$ for some integer n . Since $U^\dagger U = I$, the ideal unitary is preserved while the noise is amplified. This transformation can be applied globally or locally, i.e., U may represent either the entire circuit or a subset of gates within the circuit. The results obtained from these noise-scaled circuits are then extrapolated using a suitable technique. Let λ denote the noise-scaling factor, and infers the noise-scaled expectation value as $\langle O(\lambda) \rangle \approx f(\lambda)$, where $f(\lambda)$ is an extrapolation technique, such as linear, polynomial, or exponential function. For example, a polynomial extrapolation technique can be written as:

$$f(\lambda) = a_1 + a_2\lambda + a_3\lambda^2 + \dots + a_m\lambda^{m-1}. \quad (8)$$

Here, m denotes the number of coefficients in the polynomial function, corresponding to the polynomial of degree $m - 1$. The function is then fitted to the noise-scaled expectation values of the observable O , and the zero-noise estimate is obtained as $\langle O(0) \rangle \approx \tilde{f}(0)$, where $\tilde{f}$ denotes the fitted function. The ability to jointly optimize algorithmic and error mitigation hyperparameters while simultaneously monitoring performance metrics relies on the resource-action and Pareto analysis engine discussed in Section 3.8.

Choosing the Trotter order and number of steps in noisy settings is non-trivial, as increasing either parameter simultaneously reduces discretization error while increasing circuit depth. In realistic quantum devices, this depth amplification exacerbates decoherence and gate noise, such that beyond a certain point, higher-order formulas or finer step sizes degrade overall fidelity rather than improve it. This implies an optimal regime in which the Trotterization error and the noise-induced error are balanced. Furthermore, ZNE can partially suppress noise effects in specific regimes, thereby shifting the optimal choice of order and steps. This type of multi-layer co-design scenario directly motivates the flexible stack abstraction of AutoQuREO introduced in Section 3.2, where platform constraints on Hamiltonian simulation can be composed within a unified resource estimation and optimization workflow. Understanding the interplay among Trotter order, steps, the ZNE strategy, and hardware noise is a central objective of this study.

Previous work

Previous work on Trotterized Hamiltonian simulation has examined the balance between algorithmic Trotter error and physical noise using analytical arguments, direct simulation, and hardware benchmarks. In [67], an analytic study of TFIM and XY simulation under a gate-error model compared first, third, and higher-order Suzuki formulas and found that higher-order only becomes advantageous when gate errors are sufficiently small, with a critical scale in the range 10^{-4} and 10^{-3} . Ref. [68] modeled noisy Trotter evolution with local depolarizing noise and studied state trace distance for TFIM under second-order Trotterization, concluding that an optimal step count still exists but that a state-dependent treatment gives a less pessimistic estimate of the trade-off. In [69], Trotterized Hamiltonian simulation was benchmarked on Qiskit Aer and BlueQubit simulators, as well as IBM hardware, using first-order Trotter circuits with $t = 1$ and 5 steps, and the measured quantity was a hardware and algorithmic fidelity comparison of the output distributions. It concluded that the observed performance is governed by the trade-off between circuit depth and Trotter approximation error, with hardware noise generally being more important than the isolated Trotter error. Finally, [70] used scalable benchmark circuits built from Hamiltonian simulation to separate algorithmic and hardware effects, evaluating process fidelity for first- and second-order Trotter circuits compiled and run on IBM systems and Qiskit Aer simulations. They showed that deeper Trotterization reduces approximation error but increases noise sensitivity, with a second-order decomposition at three steps giving the best balance in their Heisenberg example.

Building on these studies, we extend the analysis of the Trotter-noise trade-off in several practically relevant ways. Most importantly, we incorporate an additional error-mitigation layer into the optimization and study the trade-off directly through observable error, rather than treating fidelity-based quantities as the main figure of merit. We also examine the optimal regime by varying both the Trotter order and the number of Trotter steps within a single framework, providing a mitigation-aware, observable-focused extension of previous analytical and benchmarking studies.

Setup

In this case study, we use AutoQuREO to generate both mitigated and unmitigated Hamiltonian simulation benchmarks to examine the trade-off between algorithmic error and physical noise, as discussed above, and to determine how the accuracy of Hamiltonian simulation can be maximized across different depolarizing error rates. We quantify this trade-off by estimating the error in a chosen observable, using the corresponding classically

computed value obtained from QuTiP [71] as the reference baseline. The Hamiltonian considered in this analysis is a 1D TFIM chain with periodic boundary conditions (PBC), which is analytically solvable, defined as:

$$H = -J \sum_{i=1}^N Z_i Z_{i+1} - h \sum_{i=1}^N X_i \quad (\text{PBC: } Z_{N+1} = Z_1) \quad (9)$$

and the observable of interest is longitudinal magnetization (i.e., along Z axis):

$$M_z = \sum_{i=1}^N Z_i \quad (10)$$

We considered a 4-qubit TFIM and, for fixed values of the Hamiltonian parameters, evolution time, and initial state, performed simulations over a range of Trotter orders and step counts at different depolarizing error rates, considering both unmitigated and mitigated cases. The noise model was defined on the $\{RZZ, RX, H, \text{Measure}\}$ gate set, with depolarizing noise applied to both single- and two-qubit gates using the same depolarizing error parameter. The simulation is performed on Qiskit Aer [72]. Table 1 shows the stack along with hyperparameters, corresponding ranges, and actions.

Table 1 Layers, hyperparameters, ranges, and resources for the quantum resource estimation scenario. The optimization minimizes the observable error while jointly analyzing the Trotter order, Trotter steps, depolarizing error rate, and error mitigation strategy.

Layers	Hyperparameters	Ranges
System definition	Transverse field (h)	[5.0]
	Coupling strength (J)	[10.0]
	Evolution time (t)	[10.0]
	Initial state	[00...0⟩]
Algorithm compilation	Trotter order	[1, 2, 4]
	Trotter steps	[200, 250, 300]
Circuit formation	Shots	[20000]
	Depolarizing error rate	[10^{-5} , 10^{-4} , 10^{-3} , 10^{-2}]
Error mitigation	Strategy	[None, ZNE Richardson] (noise scale factors: [1,3,5])

Resources
Number of qubits
Observable error
Depth

Results

From the analysis presented in Figure 9, we observe that even at very low depolarizing error rates (e.g., 1×10^{-5}), trivially increasing the Trotter order or the number of Trotter steps does not guarantee improved accuracy. This indicates that the effect of hardware noise needs to be considered as well. Further, we find that for every depolarizing error rate, there exists an optimal Trotter configuration that balances the algorithmic error ΔO_{alg} and physical error ΔO_{phys} . Beyond this optimum, the deeper circuits required for higher-order or higher-step Trotterization accumulate noise faster than ΔO_{alg} is reduced, leading to an increase in the total error. Additionally, the use case demonstrates that ZNE's effectiveness depends strongly on the accumulated circuit error. When the underlying noise becomes too high, scaling the noise further (a part of the ZNE procedure) deteriorates the extrapolated estimates rather than improving them. This trend is visible in Figure 10, where the benefit of ZNE diminishes as circuit depth grows. At higher depolarizing noise levels, as shown in Figure 9, essentially all Trotter settings produce circuits deep enough that accumulated noise dominates, causing both no-QEM and ZNE-corrected results to degrade. Together, these observations highlight the importance of co-optimizing algorithmic parameters and error-mitigation strategies, rather than treating them as independent implementation parameters.

In this use case, AutoQuREO enabled streamlining multi-objective exploration of hyperparameters for Trotter order, step count, error mitigation strategies, and error rates, as introduced in Section 3.8. The resulting Pareto fronts reveal regimes in which mitigation compensates for lower-order Trotterization, and hence can serve as guiding principles for the utility of QEM based on circuit depth. The layered adapter approach is modular and can thus be easily reused and extended to other use cases that share these layers.

Future work

A natural extension of this co-design study is to move beyond closed-system Trotterized Hamiltonian simulation and apply it to a broader class of quantum simulations where these trade-offs can be further explored. Of immediate interest is open-system simulation governed by Lindblad dynamics [73] via collision models [74]. For both

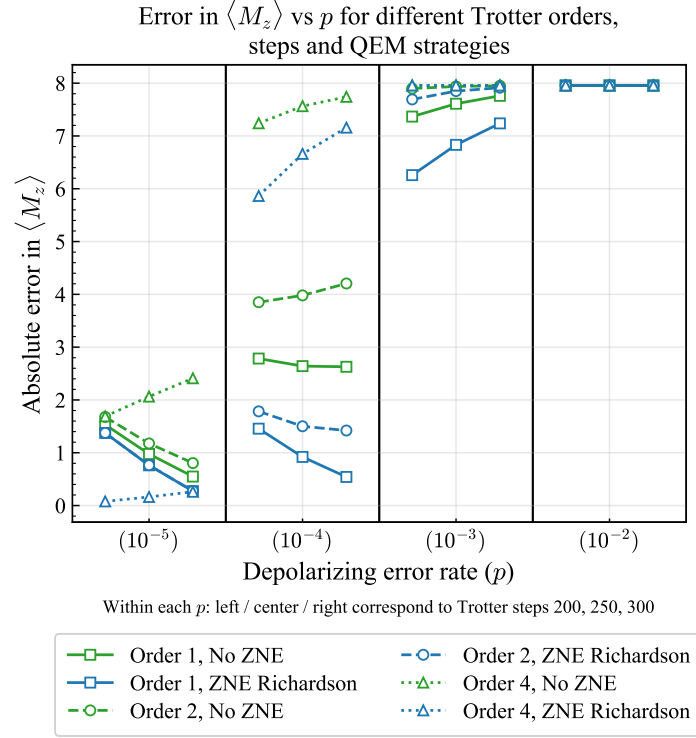


Figure 9 The absolute error in expectation value of magnetization $\langle M_z \rangle$ (for 20000 shots) for different settings observed for different depolarizing error rates in our noise model. The absolute error is calculated relative to the QuTiP result. For each point, the ticks signify Trotter steps, the marker shapes signify the Trotter order, and the color identifies the error mitigation scheme. Note: The x-axis entries (e.g., 10^{-5} , 10^{-4} , 10^{-3} , 10^{-2}) are categorical labels.

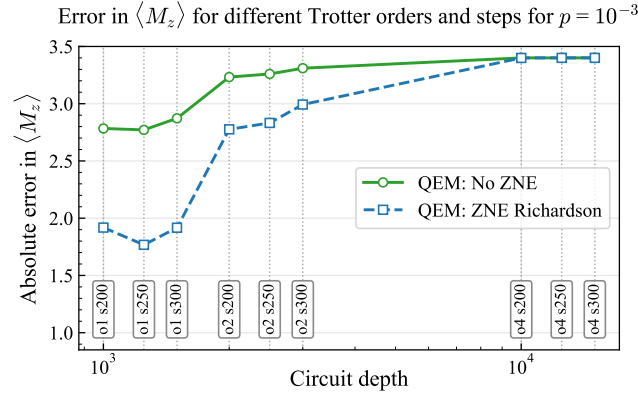


Figure 10 The variation of absolute error in expectation value of magnetization with the resulting circuit depth due to different Trotter settings for the mitigated and unmitigated cases for 10^{-3} depolarizing error rate. Note: ‘on sm’ corresponds to Trotter order n and steps m .

Hamiltonian and Lindbladian simulations, the study can be further extended to more sophisticated formulations, such as interpolated simulation schemes [75, 76], which offer additional flexibility in balancing algorithmic and physical errors and may better couple with QEM strategies. Beyond these, the framework can also be used to explore similar trade-offs in other simulation methods, such as qDRIFT [77] and quantum-trajectory-based approaches [78]. Furthermore, QEC can be combined with QEM [79, 80] and with gate decomposition using the layers developed in the subsequent sections.

In this use case, we leveraged resource estimation via compilation to demonstrate AutoQuREO’s features within an optimization workflow. As discussed in Section 3.4, synthesizing and integrating surrogate resource models would circumvent expensive circuit-level evaluations by progressively replacing learned symbolic or neural estimators. This would allow scalable predictive optimization of the use case. The next scenario will elucidate this advantage.

4.2 Scenario II

Summary

- **Novelties used:** Full-stack flexibility ($\mathcal{N}1$), library support ($\mathcal{N}2$), surrogate synthesis ($\mathcal{N}3$), and automated optimization ($\mathcal{N}4$).
- **Co-design variables:** Molecular Hamiltonian size, eigenvalue precision of iterative quantum phase estimation (iQPE), Trotter steps, gate decomposition accuracy (GridSynth ϵ), quantum error correction scheme (Steane with Reed-Muller T-teleportation, surface code), hardware noise.
- **Key insights:**
 - Symbolic LER and PER models for the Steane code gates are characterized across all-to-all and square topologies, exposing gate-wise thresholds and a routing-induced threshold window.
 - Optimization of single-qubit GridSynth decomposition fidelity is achieved by balancing synthesis accuracy against gate noise. We obtain a symbolic model matching the analytical derivation.
 - Layer-wise neuro-symbolic surrogate models compose into an iQPE stack for the Hydrazine molecule with Trotterization and GridSynth, enabling scalable resource estimation for problem sizes beyond the reach of direct compilation.
 - Propagating the optimal logical stack to quantum error correction codes yields compounding reduction in resources.

Given the inherent complexity of analyzing systems where algorithmic structure, gate decomposition, error correction, and hardware constraints interact, we present this case in three parts, each building on the outcomes of the previous, thereby highlighting the composability of surrogate resource models within the framework, as illustrated in Section 3.10.

4.2.1 Universal error correction and connectivity topology

Background

Small quantum error correction (QEC) codes, such as the Steane code [81] and Reed-Muller constructions [82], play a central role in early fault-tolerant architectures. The effective performance of QEC depends critically on the chosen gate set, assumed noise model, and resulting pseudo-threshold. While such considerations are often abstracted away in theoretical analyses, they become decisive in EFT regimes. Connectivity topology further complicates the picture. While many theoretical analyses assume all-to-all connectivity, realistic devices with thousands of qubits are constrained to sparse topologies. In EFT regimes, routing overheads can dominate error budgets and depth, necessitating explicit modeling. AutoQuREO explicitly represents these dependencies within its stack abstraction.

Setup

In this study, we measure logical error rates (LER) and physical error rates (PER) under representative noise models and symbolically annotate their dependence on code parameters and topology. This enables us to map any PER to LER and infer the pseudo-threshold analytically. Routing is performed using SABRE [83, 84], allowing comparison between routed and non-routed circuits. The resulting rates are incorporated into surrogate models that capture the interaction between topology, routing, and error correction.

The $[[7, 1, 3]]$ Steane code encodes a single logical qubit into seven physical qubits and can correct any arbitrary single-qubit error (i.e., a code distance of 3). Logical Clifford gates $\{H, S, CX\}$ are implemented transversely within this code. To achieve universality, the T gate can be implemented using the technique described in [82, 85]. Figure 11a illustrates the logical circuit for magic state ($|\overline{T}\rangle$) preparation encoded in Steane code, while Figure 11c shows the corresponding physical-level implementation of the logical circuit. Once prepared, the magic state is teleported using the gadget shown in the Figure 11b.

Table 2 shows the layers and hyperparameters for this study. In the first layer, we prepare both logical and physical circuits for characterizing the PER and LER, respectively, for all the gates $\{H, S, CX\}$. Figure 12 illustrates the recipe for constructing the PER characterization circuits. The corresponding logical circuits are obtained by omitting steps (5) and (6) in Figure 12. An example of a PER characterization circuit for H gate is shown in Figure 13. The prepared circuits are then simulated on a noisy simulator for various gate depolarizing error rates (p) and qubit connectivity topologies (all-to-all and square). In the last layer, the PER and LER are computed as the fraction of trials in which the expected output states are successfully measured.

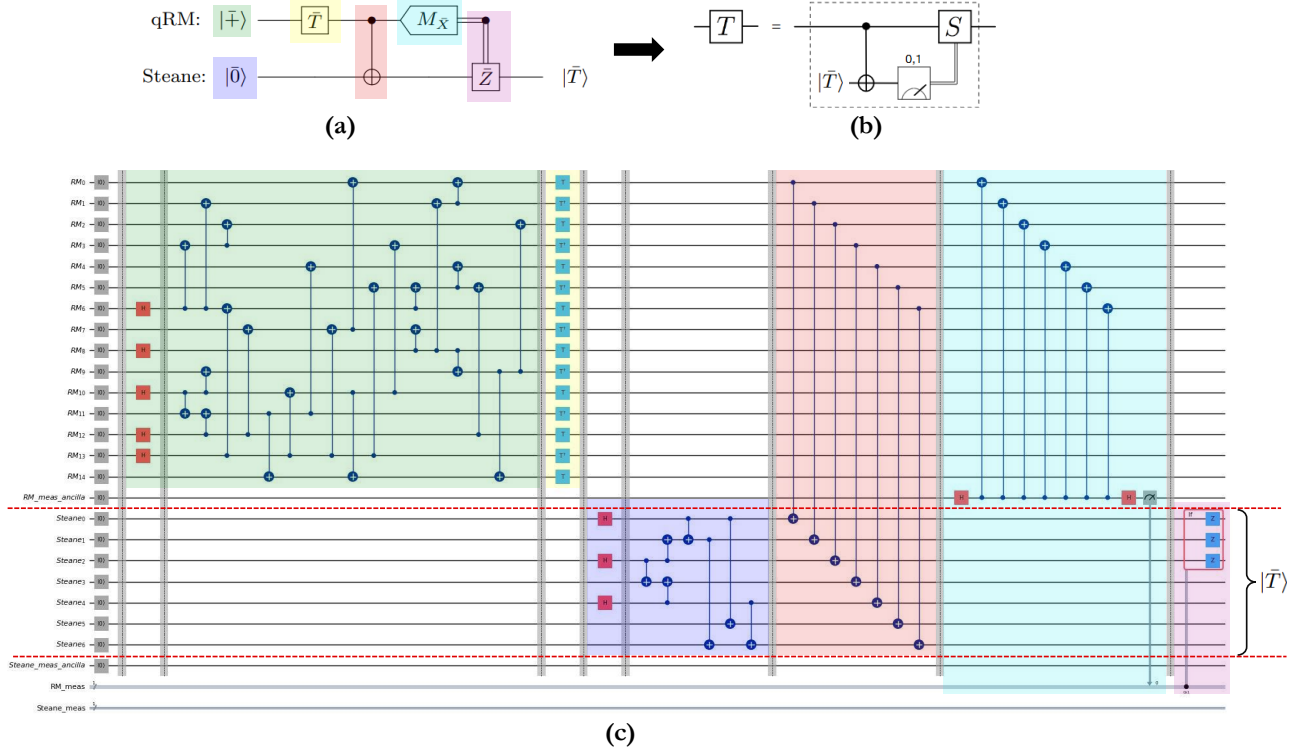


Figure 11 Universal error correction scheme using Steane code and T-teleportation from Reed-Muller (qRM) code. (a) Logical circuit for $|\bar{T}\rangle$ preparation using Reed-Muller code. (b) Circuit to implement logical T by consuming the prepared state. (c) Physical circuit corresponding to logical circuit in (a).

Table 2 Stack table for characterization of LER and PER for Clifford gates in Steane code.

Layers	Hyperparameters	Ranges
Characterization Circuits	Gates	[H, S, CX]
Simulation	Depolarizing error rate (p)	$[10^{-5}, \dots, 10^0]$
	Connectivity	[all-to-all, square]
	Shots	[10000]
LER/PER calculation	-	-

$$U3(\theta, \phi, \lambda) = e^{i\frac{\phi+\lambda}{2}} R_Z(\phi) R_Y(\theta) R_Z(\lambda) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -e^{i\lambda} \sin\left(\frac{\theta}{2}\right) \\ e^{i\phi} \sin\left(\frac{\theta}{2}\right) & e^{i(\phi+\lambda)} \cos\left(\frac{\theta}{2}\right) \end{bmatrix} \quad (11)$$

$$CU3(\theta, \phi, \lambda) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U3(\theta, \phi, \lambda) \quad (12)$$

$$U3\left(\frac{\pi}{2}, 0, \pi\right) = H, \quad U3\left(0, 0, \frac{\pi}{2}\right) = S, \quad U3\left(0, 0, \frac{\pi}{4}\right) = T, \quad CU3(\pi, 0, \pi) = CX$$

Results

Since the applied noise is unbiased, the experimentally obtained PER and LER data yield similar values for the H and S gates, as shown in Figure 14a. Consequently, a single model is sufficient for both gates, i.e., $\tilde{\epsilon}_H = \tilde{\epsilon}_S$. The red data points correspond to experimentally measured PER values, and the associated PySR model is shown by the red dashed curve. Similarly, the blue data points and curve represent the experimental PER data and the corresponding PySR model for the all-to-all topology, with a threshold value of 1.3×10^{-1} . The green

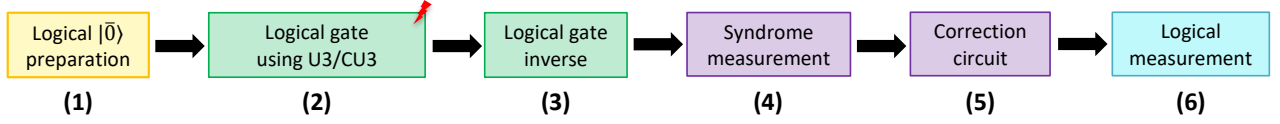


Figure 12 Flowchart for building a characterization circuit corresponding to the gates (H, S, CX, T). In step (1), Steane-encoded logical $|0\rangle$ is prepared without any errors. In step (2), we apply a noisy version of the gate to be characterized using the corresponding $U3/CU3$ implementation to selectively apply the noise ($U3/CU3$ representations for target gates are given in Equations 11 and 12). In step (3), a logical inverse gate is applied in the original operator form. For example, H gate after applying $U3$ corresponding to H in step (2). In step (4), the syndrome is extracted via the syndrome measurement circuit. In step (5), the error correction circuit inverts the detected error during syndrome measurement in step (4). In step (6), logical measurement is performed, and the data is stored for further postprocessing. Note that the noisy physical gates are applied only in steps (2), and everything else is assumed noiseless.

data points and curve correspond to the square topology. We observe a range of p values for which the LER is lower than the PER for the square topology. We refer to this as the *threshold window*, which is identified as $[2.3 \times 10^{-3}, 1.3 \times 10^{-1}]$ for this experiment. Note that the *SWAP* gate error rate is fixed at 10^{-4} .

Following the assumption given in [86], we construct an LER model for the T gate by scaling the S gate LER model by a factor of two, i.e., $\tilde{\epsilon}_T = 2 \cdot \tilde{\epsilon}_S$. For cases where $\tilde{\epsilon}_T \geq 0.5$, the resulting value can exceed unity; therefore, $\tilde{\epsilon}_T$ is clipped to a maximum value of 1.0. Figure 14b presents the experimental data and the corresponding PySR model for the CX gate, yielding a threshold value of 3×10^{-1} .

The resulting analysis informs a symbolic resource model as introduced in Section 3.4, enabling efficient reuse without repeating the analysis. These modeled error profiles are useful for applications such as estimating end-to-end circuit fidelities and integrating with other transpilation steps. In the next section, we leverage these results to study the co-design of gate decomposition and logical error rates, demonstrating how hardware-aware error modeling can guide the identification of optimal operating regimes.

Future work

Looking forward, the models can be utilized in a comparative study across quantum error detection (QED) codes (e.g., Iceberg-code [87]), partial fault tolerance (e.g., STAR architecture [88], dirty-qubits [89], CliNR [90]), and full fault tolerance (e.g., surface code [91], quantum low density parity code [92]). These approaches introduce distinct trade-offs between logical error rates, code size, circuit depth, and hardware constraints. Symbolic annotations from existing QRE tools, including those developed in [11] and [86], can be integrated seamlessly, demonstrating interoperability with already developed QRE primitives. Such experiments give insight into practical implementations of quantum error correction strategies.

4.2.2 Gate decomposition accuracy and logical error rate

Background

Gate decomposition is a critical layer bridging algorithmic intent and hardware execution. While near-term devices tolerate relatively coarse approximations, fault-tolerant regimes impose stringent accuracy requirements that dramatically increase circuit depth. Understanding the optimal balance between decomposition accuracy and the accumulation of logical errors is therefore essential. In noisy settings, higher decomposition accuracy does not monotonically improve overall fidelity. Instead, a noisy optimal zone emerges in which decomposition error and noise-induced error balance. This phenomenon exemplifies why AutoQuREO treats decomposition fidelity, logical error rate, and circuit depth as simultaneously co-optimized resources rather than isolated post-compilation metrics, as formalized in Section 3.8. A similar study was conducted in [93] for only T gate errors. Using AutoQuREO, we model a universal gate set and identify optimal accuracy regimes as a function of noise strength and thereafter broaden this study to incorporate quantum error correction and algorithms.

Here, we study this trade-off for single-qubit unitaries using GridSynth [94] for synthesis. The GridSynth algorithm is designed to synthesize arbitrary R_z rotations up to an approximation error of ϵ (defined in operator norm as the difference between the original and approximated unitary) using a sequence of H , S , and T gates. The T gate count scales as $3 \log_2 \left(\frac{1}{\epsilon} \right) + O \left(\log \left(\log \left(\frac{1}{\epsilon} \right) \right) \right)$ with an expected runtime of $O \left(\text{polylog} \left(\frac{1}{\epsilon} \right) \right)$. To synthesize an arbitrary single-qubit unitary, one first computes its Euler-angle decomposition. Then, via conjugation with

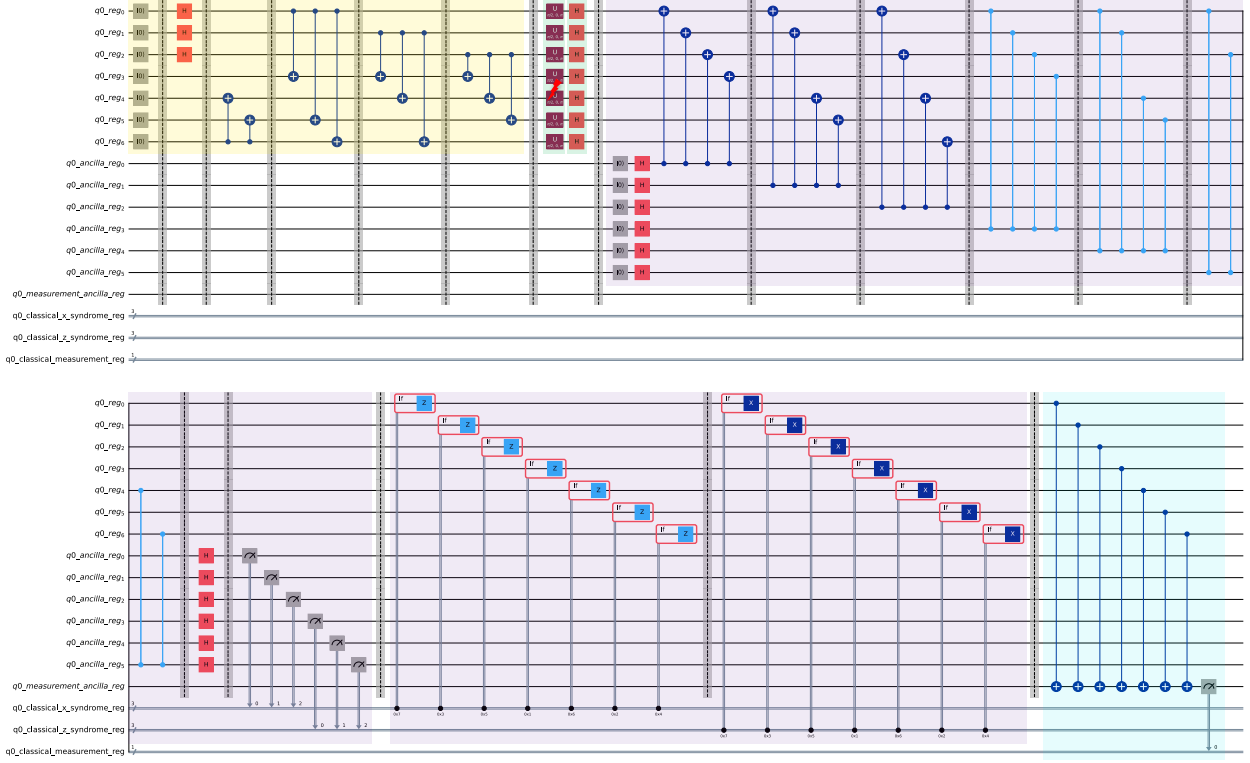


Figure 13 Physical-level circuit for characterizing the LER for H gate in the Steane code, constructed via Figure 12: encoded $|0\rangle$ preparation, a noisy H (applied as $U3$) and its logical inverse, syndrome measurement, error correction, and logical measurement, with only the H gate made noisy.

Table 3 The left panel presents the stack table for gate decomposition accuracy and LER co-design, whereas the right panel shows the corresponding resource table. For AutoQuREO-based optimization, the actions correspond to maximizing the average state fidelity for each depolarizing error rate.

Layers	Hyperparameters	Ranges	Resources
Unitary generation	Samples	[100]	Number of qubits
GridSynth decomposition	Accuracy (ϵ)	$[10^{-10}, \dots, 10^{-1}]$	Average state fidelity
Simulation	Depolarizing error rate (p)	$[10^{-8}, \dots, 10^{-4}]$	

Clifford gates, an equivalent circuit consisting of three R_z rotations is obtained. Finally, GridSynth approximates each R_z individually. In this case, the T gate count scales as $9 \log_2 \left(\frac{1}{\epsilon} \right) + O \left(\log \left(\log \left(\frac{1}{\epsilon} \right) \right) \right)$ [94].

Setup

To characterize GridSynth, we begin by sampling 100 Haar-random single-qubit unitaries. Each unitary is then synthesized using GridSynth for various accuracies ϵ . The resulting sequences of H , S , and T gates are applied to the initial state $|0\rangle$, and run on a noisy density-matrix simulator. We use a logical-error model with depolarizing noise of varying strength p . From the final density matrices, ρ , the state fidelity is computed with the initial state $\rho_0 = |0\rangle\langle 0|$, as $F = \langle 0 | \rho | 0 \rangle$. Table 3 summarizes the stack, hyperparameters, corresponding ranges, and resources.

Results

Figure 15a shows the infidelity $1 - F$ (left y-axis) and circuit depth (right y-axis) vs GridSynth accuracy (ϵ) graphs for various depolarizing error rates (p). We clearly observe that the depth increases linearly with $-\log_{10}(\epsilon)$. A PySR symbolic regression model infers the average depth as $D \approx 75 \log_{10} \left(\frac{1}{\epsilon} \right)$. Since the noise model is unbiased, the average statistics capture random initial states instead of $|0\rangle$.

AutoQuREO's optimization feature filters configurations based on specified hyperparameter and resource actions. For this study, we set the actions to maximizing the average state fidelity for each depolarizing error rate. These resulting filtered configurations are marked with a star in Figure 15a for all values of p . These minima indicate

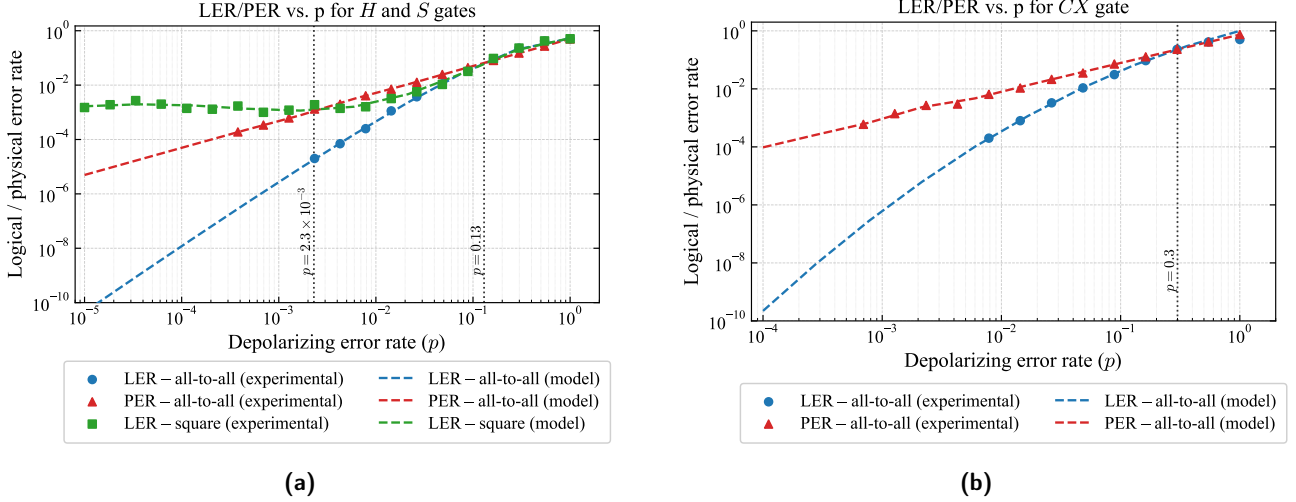


Figure 14 Experimental characterization data and the corresponding PySR models for H , S and CX gates. **(a)** Experimental data and model of H and S gates for all-to-all and square topology. For all-to-all connectivity, LER becomes lower than PER for $p = 1.3 \times 10^{-1}$. For square topology, $SWAP$ gates are injected, and the error rate for $SWAP$ gates was set to 10^{-4} . The green curve corresponding to square topology shows a *threshold window* from 2.3×10^{-3} to 1.3×10^{-1} , i.e., for p in this range, LER is lower than PER. **(b)** Experimental data and model of CX gate for all-to-all topology. The threshold is estimated to be 3×10^{-1} .

the existence of an optimal ϵ across all noise levels that minimizes infidelity (thereby maximizing fidelity). These filtered configurations are then used to model the optimal decomposition accuracy, ϵ_{opt} , as a function of p . This illustrates the iterative interplay between optimization and modeling in AutoQuREO. A full sweep over the decomposition accuracy is first optimized against noise to isolate the optimal Pareto points, these points are then modeled with PySR into a closed-form relation, and the resulting equation is reused as the resource model for the decomposition layer in the full-stack estimation. Optimization is therefore not a one-shot filtering, but a step that feeds reusable surrogate models back into the estimation loop for efficient full-stack DSE. Only the final selected configuration needs compilation. We use two methods to derive ϵ_{opt} :

1. Using PySR for symbolic regression (shown as the red curve in 15b), we obtain:

$$\epsilon_{\text{opt}} \approx 2.28\sqrt{p} + 162.43p \quad (13)$$

2. By analytically deriving the model (shown as the blue curve in 15b and derived later in this section), we obtain:

$$\epsilon_{\text{opt}} \approx 2.85\sqrt{p} \quad (14)$$

Comparing these two methods for model generation, we observe that the PySR model closely matches both the analytical model and the simulation results, while requiring significantly less time and effort. In contrast, the manual analytical method is non-trivial and requires expertise.

Since we assume a depolarizing logical error model and perform direct simulations, the resulting plots show exact infidelities. However, when a QEC code such as Steane is used, density-matrix simulation of Steane-encoded circuits becomes impractical due to the large number of T gates and their implementation using a Reed-Muller code. Consequently, it is essential to develop a model that estimates fidelity as a function of ϵ and p . To this end, we empirically create a fidelity estimation model, denoted by F_{est} , using exact fidelity data as follows:

$$F_{\text{est}} = \frac{1}{2} [1 + K(1 - 2\epsilon^2)] \quad (15)$$

where K is the estimated success probability (ESP) [95], defined as:

$$K = \prod_{i=1}^m (1 - \tilde{\epsilon}_{g_i})^{n_i} \quad (16)$$

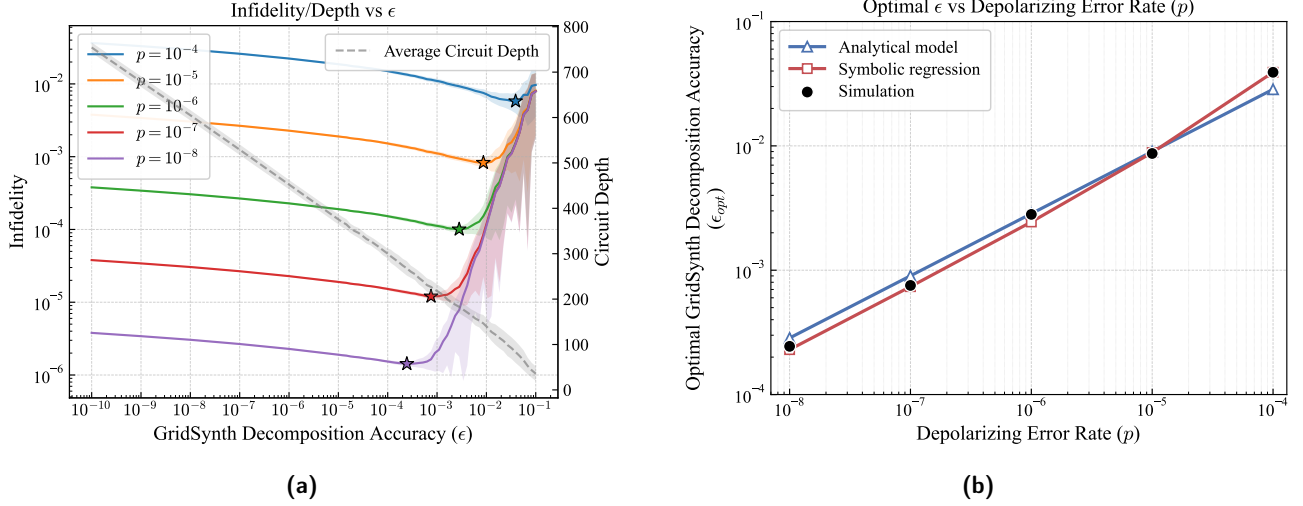


Figure 15 Performance metrics of GridSynth decomposition under depolarizing noise. **(a)** Infidelity ($1 - F$) and circuit depth vs decomposition accuracy (ϵ) for various depolarizing error rates (p). Star markers indicate the filtered configuration after AutoQuREO optimization. **(b)** Optimal decomposition accuracy (ϵ_{opt}) vs depolarizing error rate (p). Red curve shows the PySR model ($\epsilon_{\text{opt}} \approx 2.28\sqrt{p} + 162.43p$) generated using the simulation data. Blue curve shows the analytical model ($\epsilon_{\text{opt}} \approx 2.85\sqrt{p}$).

Here we assume that $G = \{g_1, g_2, \dots, g_m\}$ is the gate set, and $N = \{n_i \mid n_i \text{ is counts of } g_i\}$ is the set of counts for each gate g_i in a given circuit. $\tilde{\epsilon}_{g_i}$ is the LER, and it is a function of p in general.

For this study, we are assuming $G = \{H(g_1), S(g_2), T(g_3)\}$ and $\tilde{\epsilon}_H = \tilde{\epsilon}_S = \tilde{\epsilon}_T = p$. Therefore,

$$K = \prod_{i=1}^3 (1 - \tilde{\epsilon}_{g_i})^{n_i} = \prod_{i=1}^3 (1 - p)^{n_i} = (1 - p)^{\sum n_i}.$$

Substituting the values for the gate counts for the gate set G as defined in Equation 18,

$$K \approx (1 - p)^{75 \log_{10}(\frac{1}{\epsilon})} = e^{-\frac{75}{\ln(10)} \ln(\epsilon) \ln(1-p)} = \epsilon^A,$$

where $A = -\frac{75}{\ln(10)} \ln(1 - p)$. Since $0 < p < 1$, $\ln(1 - p) < 0$, $\therefore A > 0$. Substituting in (15), we get $F_{\text{est}}(\epsilon) = \frac{1}{2}[1 + \epsilon^A(1 - 2\epsilon^2)]$.

Analytical model derivation

To calculate ϵ_{opt} theoretically, we maximize F_{est} w.r.t ϵ . Therefore,

$$F'_{\text{est}}(\epsilon) = \frac{1}{2} [A\epsilon^{A-1} - 2(A+2)\epsilon^{A+1}].$$

From $F'_{\text{est}}(\epsilon) = 0$, we obtain,

$$\epsilon_{\text{opt}} = \sqrt{\frac{A}{2(A+2)}}. \quad (17)$$

If $p \ll 1$, then $\ln(1 - p) \approx -p$ and $A \approx 32.57p \ll 1 < 2$. Therefore, $\epsilon_{\text{opt}} \approx \sqrt{\frac{A}{4}} = 2.85\sqrt{p}$.

Substituting ϵ_{opt} in $F_{\text{est}}(\epsilon)$, the analytical maximum fidelity is

$$F_{\text{est}}^{\text{max}} = \frac{1}{2} \left[1 + \left(\frac{A}{2(A+2)} \right)^{A/2} \frac{2}{A+2} \right].$$

For $A \ll 1$, using $\frac{A}{2(A+2)} \approx \frac{A}{4}$ and the further approximation $\frac{2}{A+2} \approx 1$, we get

$$\begin{aligned}
 F_{\text{est}}^{\text{max}} &\approx \frac{1}{2} \left[1 + \left(\frac{A}{4} \right)^{\frac{A}{2}} \right] \\
 &= \frac{1}{2} \left[1 + \exp \left(\frac{A}{2} \ln \left(\frac{A}{4} \right) \right) \right] \\
 &\approx \frac{1}{2} \left[1 + 1 + \frac{A}{2} \ln \left(\frac{A}{4} \right) \right] (\because e^x \approx 1 + x) \\
 &\approx 1 + \frac{A}{4} \ln \left(\frac{A}{4} \right) \\
 &\approx 1 + 8.14p \ln(8.14p)
 \end{aligned}$$

As p becomes small, ϵ_{opt} scales like $\sqrt{p}$, and the minimum infidelity $1 - F_{\text{est}}^{\text{opt}}$ scales like $p |\ln p|$ (up to constants).

Note that the above analysis assumes that all the gates in G have identical LERs. However, the T gate generally exhibits a higher LER because it is implemented using techniques such as magic-state distillation [96] and magic-state cultivation [86]. In this study, we employ the Reed-Muller code described in Section 4.2.1. Additionally, we create PySR models for H , S , and T gate counts as functions of GridSynth ϵ to obtain:

$$n_H \approx 30 \log_{10} \left(\frac{1}{\epsilon} \right), \quad n_S \approx 15 \log_{10} \left(\frac{1}{\epsilon} \right), \quad n_T \approx 30 \log_{10} \left(\frac{1}{\epsilon} \right) \quad (18)$$

Using Equations 16, 18, and the relation $\tilde{\epsilon}_H = \tilde{\epsilon}_S \neq \tilde{\epsilon}_T$, the study can be extended to a more realistic setting. For example, we may assume $\tilde{\epsilon}_T = p$ and $\tilde{\epsilon}_H = \tilde{\epsilon}_S = 10^{-2}p$ (assuming that T gate has an LER two orders of magnitude higher than that of H and S gates), and derive new models for ϵ_{opt} using both PySR and the analytical method.

Integration with Steane code

Next, we extend this study by incorporating LER models for the Steane code single-qubit gates $G = \{H, S, T\}$ from Section 4.2.1. We assume that the PER for these gates is approximately p , and that the corresponding LERs are functions of p , i.e., $\tilde{\epsilon}_{g_i} = f_i(p)$ for all $g_i \in G$. To estimate the fidelities, we use the empirical model given in Equation 15, together with the ESP formula in Equation 16. Figure 16 shows the infidelity of QEC-encoded and unencoded circuits under all-to-all and square qubit connectivity topologies. As observed, for all-to-all connectivity, the encoded circuits achieve lower infidelities than their unencoded counterparts. In contrast, under square connectivity, the encoded circuits become impractical due to the accumulation of $SWAP$ gate errors. Thus, the square connectivity would therefore have practical relevance over all-to-all connectivity in regimes with significant lower physical noise level and scalable quantum processors.

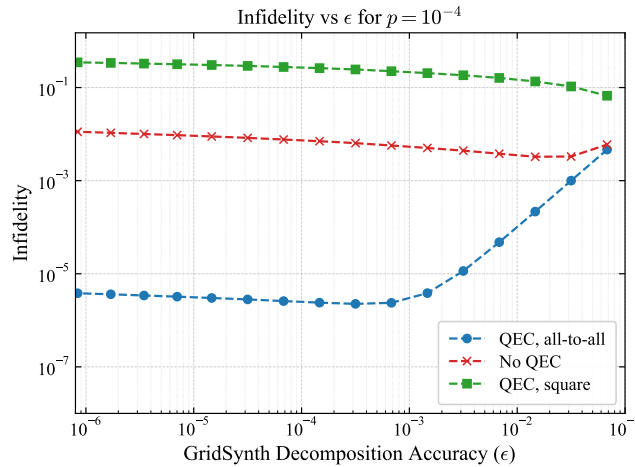


Figure 16 Infidelity ($1 - F$) vs GridSynth decomposition accuracy (ϵ) for physical gate depolarizing error rate $p = 10^{-4}$. We used the LER models generated in Section 4.2.1 and assume $\tilde{\epsilon}_H = \tilde{\epsilon}_S$, $\tilde{\epsilon}_T = 2 \cdot \tilde{\epsilon}_H$ with clipping the maximum value to 1.0.

Future work

The analysis presented in this section can be extended to other single-qubit unitary synthesis algorithms, such as the Solovay-Kitaev [97], TraSyn [93], and Morisaki-Sano-Akibue algorithm [98]. Furthermore, this framework can be extended to multi-qubit unitaries via decomposition techniques, including quantum Shannon decomposition [99], QSeed [100], flag decomposition [101], and can be integrated with QEC models to capture compounding effects.

4.2.3 Ground-state energy estimation with iterative quantum phase estimation

Background

Early fault-tolerant (EFT) [102, 2] regimes occupy an increasingly relevant middle ground between noisy intermediate-scale quantum (NISQ) devices and fully fault-tolerant quantum computers (FTQC). NISQ devices have qubit counts potentially beyond classical simulation limits (in the range of 50-100 qubits), with physical noise levels that enable QEC codes to demonstrate below-threshold operation (typically a single logical qubit memory or basic logical gates). Thus, full QEC encoding of logical qubits and the required operations for algorithms is not possible, limiting their applicability to low-depth unencoded circuit use cases, such as variational algorithms implemented by parametric quantum circuits and trained iteratively via a hybrid quantum-classical optimization loop. FTQC refers to a regime in which the above issues have been addressed, so that users have a sufficient number of high-fidelity, error-corrected qubits at their disposal, with coherence maintained beyond the required circuit execution duration. FTQC algorithms are conventional ones with well-motivated applications and theoretical complexity guarantees, such as quantum factoring, quantum phase estimation, and linear equation solvers. EFT bridges these two regimes via principled approaches that would enable a smooth transition. These include incorporating low-ancilla overhead QEC codes, partial error correction, post-selection based on error detection, circuit cutting and knitting, non-uniform qubit basis calibration, error mitigation via circuit-level and pulse-level protocols, randomized algorithms, and low-ancilla iterative algorithms, among others. Thus, even in modest problem instances, EFT stacks already combine algorithmic structure, small-scale error correction, discrete gate synthesis, and noisy backends with connectivity topologies. Manually reasoning about their coupled resource trade-offs rapidly becomes infeasible.

Setup

In this section, we consider an EFT algorithm, in particular, the iterative quantum phase estimation (iQPE) [103, 104], applied to the electronic ground-state energy estimation (GSEE) of the N_2H_4 (Hydrazine) molecule. We import the Hamiltonian (H) expressed as a sum of Pauli strings (Equation 19) from PennyLane datasets [16]. For a molecule not in the database, first a quantum chemical electronic structure calculation using Psi4 [105] is run for the molecule in STO-3G basis set. Then the Psi4 executes SCF, MP2, CISD, CCSD, and FCI methods [105] to obtain the molecular integrals. The molecular Hamiltonian (in terms of one- and two-electron integrals) is then converted to a second-quantized Fermionic operator. Following this, the Fermionic ladder operators are mapped to qubit Pauli operators via the Jordan-Wigner transform, eventually yielding the qubit Hamiltonian expressed as a sum of Pauli strings.

We specifically focus on symbolically inferring the gate-count resource, incorporating estimates from error correction and decomposition from the previous section, for a full-stack QRE. We also extend this study to estimate FTQC resources by replacing Steane code with surface code. Table 4 summarizes the stack, hyperparameters, corresponding ranges for Steane code (with T-teleportation from Reed-Muller code) and surface code.

$$H = \sum_{i=1}^L c_i P_i, \quad \text{where } P_i \in \{I, X, Y, Z\}^{\otimes n}, \quad c_i \in \mathbb{R}, \quad n = 28 \text{ for } \text{N}_2\text{H}_4. \quad (19)$$

Its matrix exponential (setting $t = 1$) is the time-evolution unitary $U = e^{-iH}$. The qubit Hamiltonian is then diagonalized to find the smallest algebraic eigenvalue corresponding to the ground-state energy and the associated eigenvector:

$$H |\psi_0\rangle = E_0 |\psi_0\rangle, \quad E_0 = \min_{\lambda \in \sigma(H)} \lambda \quad (20)$$

The ground-state eigenvector $|\psi_0\rangle$ is prepared directly from the classically computed amplitudes $\{c_k\}$:

$$|\psi_0\rangle = \sum_{k=0}^{2^n-1} c_k |k\rangle \quad (21)$$

Table 4 Full-stack resource estimation table showing the layers, hyperparameters, and corresponding ranges. Hyperparameters for the Steane code (T from Reed-Muller code) and the surface code implementation are listed under their respective columns. The first two layers (Preprocessing and iQPE) use identical hyperparameters and ranges for both implementations, whereas the third (first-order Trotterization + GridSynth) and fourth (Error Correction) layers use implementation-specific hyperparameter sets.

Layers	Steane code (T from Reed-Muller code)		Surface code	
	Hyperparameters	Ranges	Hyperparameters	Ranges
Preprocessing	Bond length (Å)	[1.446]	Bond length (Å)	[1.446]
iQPE	Eigenvalue precision	[10]	Eigenvalue precision	[10]
Trotterization + GridSynth	Trotter steps	[5]	Trotter steps	[5]
	GridSynth accuracy	$[2.5 \times 10^{-4}]$	GridSynth accuracy	$[7.2 \times 10^{-7}]$
Error Correction	Depolarizing error rate	$[10^{-4}]$	Physical error rate	$[10^{-4}]$
			Code distance (data)	[11]
			No. of factories	[5]
			L1 distillation code distance (factory)	[19]
			L2 distillation code distance (factory)	[31]
			Cycle time (μs)	[1]

Results

Algorithm resources

In conventional QPE, an n -qubit control register is prepared in a superposition and entangled with a target eigenstate via controlled powers of a unitary operator, followed by an inverse quantum Fourier transform (QFT) to extract the phase, enabling parallel estimation of multiple phase bits. The iterative quantum phase estimation (iQPE) [103] is a resource-efficient QPE variant that replaces the multi-qubit control register with a single ancilla qubit reused across multiple rounds to estimate the phase bitwise sequentially. As shown in Figure 17, each iteration applies a controlled- U^{2^k} operation, followed by a phase correction based on previously measured bits and a Hadamard rotation prior to measurement. The measurement outcome probabilistically determines the k -th bit of the phase, and the process is repeated for increasing powers of U . This structure eliminates the need for a full QFT, making iQPE particularly suitable for EFTs constrained by qubit and circuit coherence, at the cost of higher runtime than QPE.

We are interested in modeling the gate counts of iQPE with respect to the eigenvalue precision ep and the qubit size of the input unitary qu . For example, from visual inspection of these circuits, qubit count scales as $qu + 1$, state preparation and measurement operator scales as ep , Hadamard scales as $2 * ep$, and reset operators scale as $(qu + 1) * ep$. The scaling for controlled unitary and phase gates is non-trivial, with $\sum_{i=0}^{ep-1} 2^i$ and $\sum_{i=0}^{ep-1} i$ if $ep > 1$, respectively. AutoQuREO's surrogate modeling enables us to verify whether these intuitions are correct and to automate this resource analysis without requiring expertise in analytical complexity theory. PySR-based symbolic regression infers the following symbolic resource estimates from compilation data of $ep = [1, 6]$ and $qu = [1, 7]$ as shown in Box 1.

Box 1: Gate resources for iQPE

```

qb          : qu + 1.0000001
reset       : qu*ep + ep
state_preparation : ep
cu_in      : 1.9999988**ep - ep/ep
h          : ep + ep
measure    : ep
p          : -0.500011*ep - 0.0002529222*ep*(-0.20524421) + ep/((2.0000248/ep))

```

We note that the surrogate model can replicate all the trivial equations and suggest alternate forms for the non-trivial ones within specified error bounds. Demonstratively, the inferred equation for the controlled unitary,

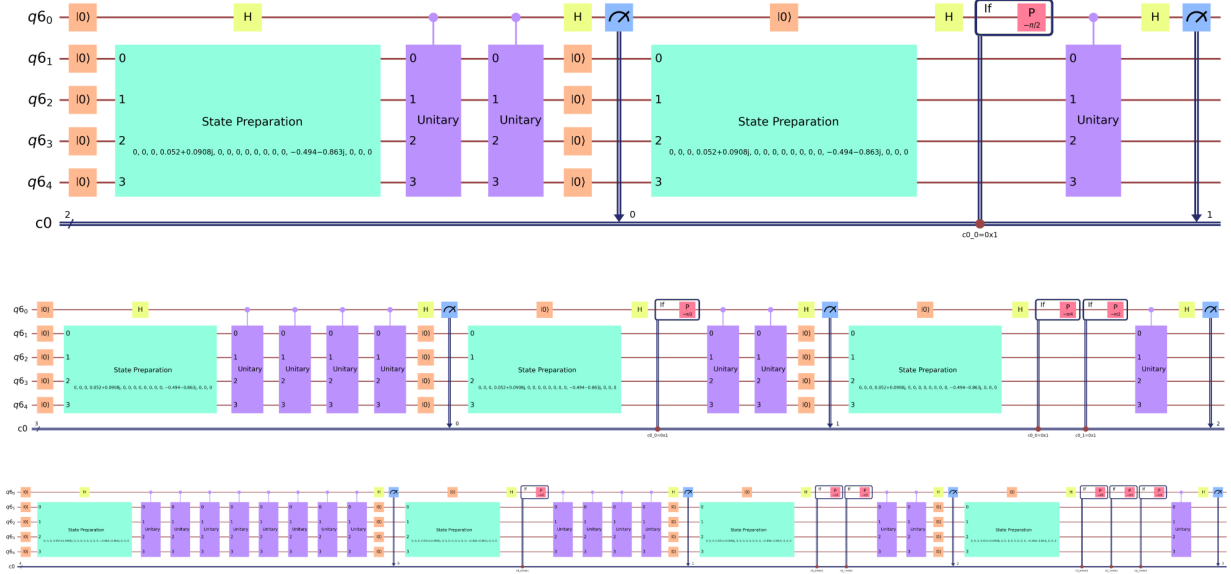


Figure 17 Quantum circuits for iterative quantum phase estimation (iQPE) algorithm for precision of 2, 3, and 4 bits. Subsequent circuits and gate colors highlight the scaling patterns.

$2^{ep} - 1$, is simpler than our analytical benchmark. Characteristics bloat from genetic programming (e.g., the `ep/ep` instead of 1) and convergence error from simulated annealing (e.g., the `1.0000001`) can be further fine-tuned using PySR’s parameters (e.g., `parsimony`) within Optuna to refine the surrogate model. These equations, however, are within acceptable modeling errors for our representative case study.

Decomposition resources

The controlled unitary gates (`cu_in`) in iQPE require explicit decomposition to map to the EFTQC and FTQC stack. We assume an initial $|0\rangle^{\otimes 28}$ state having a non-zero overlap with the ground-state. Since the Hamiltonian (H) is written in Pauli basis (Equation 19), the time evolution operator can be implemented by exponentiating the individual Pauli terms and constructing the corresponding controlled gates. Here, we employ first-order Trotterization, as described in Equation 5, with r Trotter steps to estimate gate resources. Let x_i , y_i , and z_i denote the number of X -, Y -, and Z -Pauli operators, respectively in the Pauli term P_i . The Pauli weight of P_i is then given by, $w_i = x_i + y_i + z_i$, and `sum_paulis()` denotes the summation over all Pauli terms in the Hamiltonian. The resulting gate counts are summarized in Box 2.

Box 2: Gate resources for first-order Trotterization

```

cx : sum_paulis(2 * (w_i - 1)) * r * cu_in + 2 * L * r * cu_in
h  : sum_paulis(2 * (x_i + y_i)) * r * cu_in
s  : sum_paulis(y_i) * r * cu_in
s_dg: sum_paulis(y_i) * r * cu_in
rz : 2 * L * r * cu_in

```

Thus, the iQPE gate resources can now be expressed in terms of single- and two-qubit gates. In situations where the Pauli basis representation is not available, we can incorporate the block-ZXZ improvement [106] of quantum Shannon decomposition (QSD) [99, 107] for multi-qubit unitaries synthesis. The associated resources are $\frac{9}{16} \cdot 4^n - \frac{3}{2} \cdot 2^n$ for CX gates and $\frac{21}{16} \cdot 4^n - \frac{3}{2} \cdot 2^n$ for rotation gates (with ratio of `rz:ry` gates being 2 : 1). We can thereafter use the relation $R_y(\theta) = SHR_z(\theta)HS^\dagger$ to convert rotations to `rz` counts.

In this case, after Trotterization, the `rz` gates are decomposed using GridSynth. Using the characterization data of Steane code from Section 4.2.1, and the PySR modeled Equations 13 and 18 for optimal decomposition settings, the gate counts for noisy hardware (`depol`) are calculated as shown in Box 3. The function `characterization_data()` corresponds to the experimental data of 14a for Steane code, and to the surface code logical error rate model, $p_L = 0.1 \left(\frac{p}{0.01} \right)^{\frac{d+1}{2}}$ from [108, 109].

Box 3: Gate resources for hardware-optimized GridSynth

```
p = characterization_data(depol)
h : ceiling(30*log(1/(2.28*sqrt(p) + 162.43*p))/log(10))
s : ceiling(15*log(1/(2.28*sqrt(p) + 162.43*p))/log(10))
t : ceiling(30*log(1/(2.28*sqrt(p) + 162.43*p))/log(10))
```

Thus, the trade-off study from the previous section is integrated into these broader application settings. For brevity, we omit the intermediate step of replacing `rz` with this cost model.

Multi-layer model comparison

The symbolic estimates above were obtained layer by layer and then composed manually. This becomes cumbersome motivating a single surrogate that directly maps to the final resource. The neuro-symbolic pipeline of Section 3.4 is well suited to maintain inference speed and interpretability for downstream analysis in such use cases.

To demonstrate this, we consider a multi-layer stack for the GSEE of the Hydrazine molecule (28 qubits). The quantum circuit composes the iQPE algorithm layer (thus, 29 logical qubits), first-order Trotterization with 5 steps, and GridSynth decomposition at an accuracy of 10^{-10} . As an extrapolation test, the models are trained with compilation data for an eigenvalue precision in the range $[2, 12]$, while the test inference is done for the range $[12, 30]$ for 400 sampled configurations.

The surrogate is synthesized by the two-stage neuro-symbolic pipeline. A NEAT network is first evolved on the compilation data as a sub-symbolic approximator, and PySR then distills the trained network into closed-form symbolic resource estimates. A comparison of the two models on inference cost and predictive confidence is shown in Figure 18. As shown in Figure 18a, the distilled symbolic model evaluates substantially faster than the standalone NEAT surrogate, since inference reduces to substituting hyperparameters in a formula rather than a full network forward pass. Figure 18b shows that this speedup incurs no meaningful loss of accuracy, as the RMS error of the distilled model matches closely with the NEAT surrogate across the extrapolated regime. The distillation adds a one-time modeling overhead of about 30 minutes, in exchange for interpretable equations. The benefit of the neuro-symbolic approach is therefore twofold, combining the low inference cost and transparency of a symbolic estimate with the predictive quality of the evolved network.

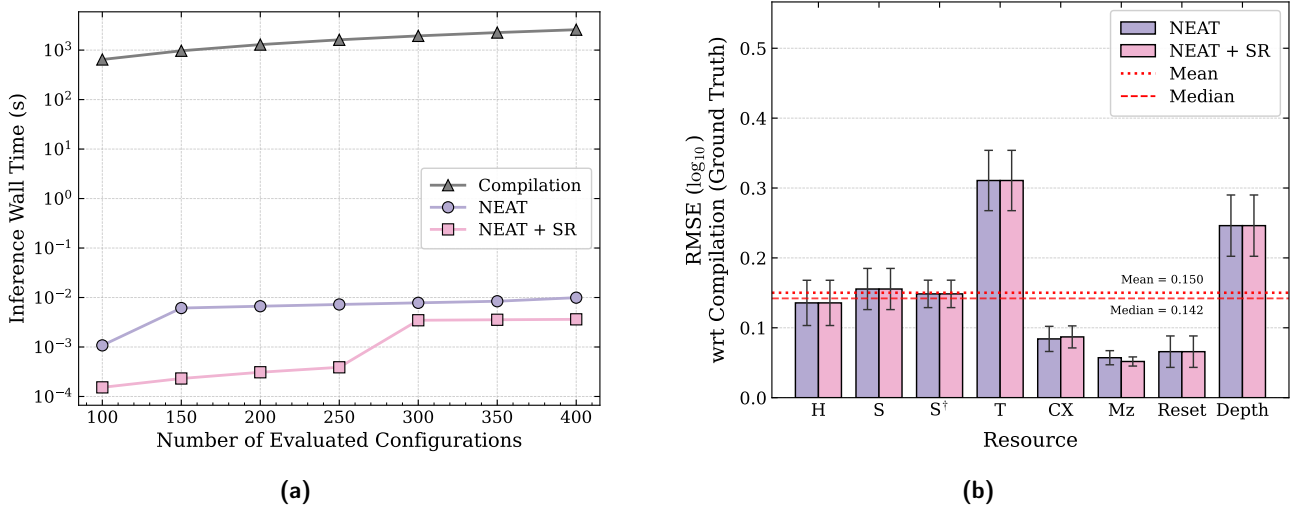


Figure 18 Performance metric comparison of surrogate models for iterative quantum phase estimation resource prediction. The two models considered are, (i) neuro-evolution using augmented topologies (NEAT), and (ii) subsequent distillation of the NEAT model with PySR (modeling configurations available in the repository). **(a)** Inference time on extrapolated test data. **(b)** RMS error on extrapolated test data. Note that the NEAT + SR model takes an additional ≈ 30 min modeling time, while being interpretable for formal analysis.

These surrogate estimators follow the broader algorithmic profiling methodology described in Sections 3.4 and 3.6, enabling interpretable scalable extrapolation beyond directly compilable problem sizes.

Algorithm and decomposition resources under noise

The optimal GridSynth accuracy derived in Section 4.2.2 was obtained for a single R_z rotation, relating ϵ_{opt} to the underlying noise as shown in Figure 15, and distilled into Equation 13. We now propagate this single-gate result through the iQPE stack for the Hydrazine molecule. The Trotterized controlled unitaries expose the R_z rotations which are thereby synthesized by GridSynth. Choosing the noise-matched ϵ_{opt} rather than a fixed accuracy compounds the per-gate saving into a substantial reduction of the total gate count. In this experiment we do not commit to a particular PER-to-LER mapping induced by a specific QEC code, but instead vary the underlying logical noise p directly, so that the benefit is characterized independently of the error-correction choice. Figure 19a sweeps the problem size via the eigenvalue precision at a fixed noise level of $p = 10^{-4}$ (used to compute ϵ_{opt}). The gap between the optimized and baseline curves widens with precision, since the number of Trotterized R_z rotations, and hence the number of GridSynth calls that each benefit from the coarser ϵ_{opt} , grows. Figure 19b sweeps the underlying logical noise at a fixed eigenvalue precision of 10. As the noise increases, ϵ_{opt} coarsens as $\sqrt{p}$, so fewer H , S , and T gates are needed per rotation, and the total gate count of the optimized stack drops accordingly. The single-gate decomposition trade-off scales up to a decisive full-stack resource saving, which would be impractical to establish by directly compiling the stack at every configuration.

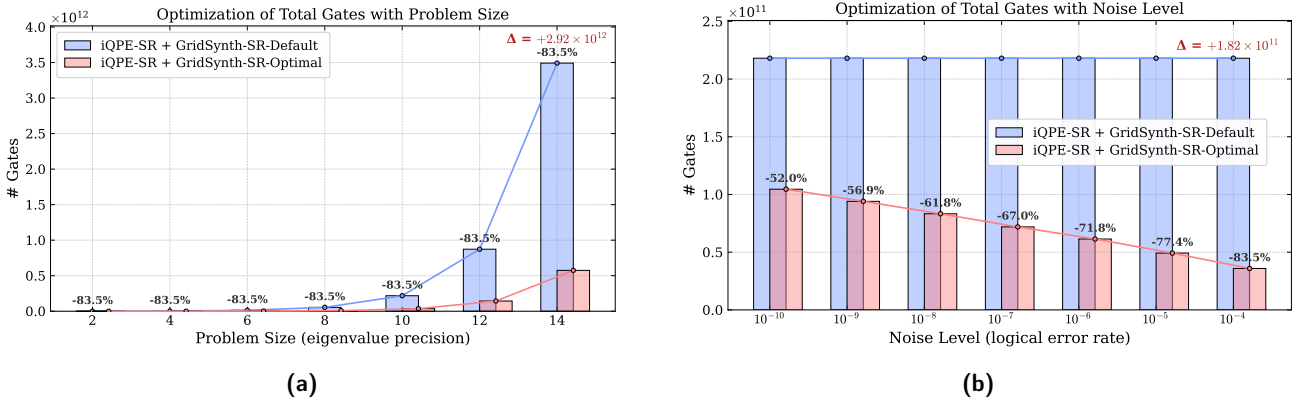


Figure 19 Full-stack gate-count saving from the optimal GridSynth accuracy ϵ_{opt} versus a fixed $\epsilon = 10^{-10}$ baseline. **(a)** Total gate count vs eigenvalue precision at a fixed LER $p = 10^{-4}$. The absolute gap widens with precision, while the relative saving stays nearly constant at $\approx 83.5\%$. **(b)** Total gate count vs LER p at a fixed eigenvalue precision of 10. As p increases, ϵ_{opt} coarsens, so fewer H , S , and T gates are synthesized per rotation and the saving grows, reaching $\approx 83.5\%$ at $p = 10^{-4}$.

4.2.4 Extension to fault-tolerant regimes

Steane resources

Next, we need to count the number of physical gates based on the Steane code and T-teleportation from Reed-Muller code as detailed in Section 4.2.1. This is shown in Box 4.

Box 4: Gate resources for QEC (Steane code, T-teleportation from Reed-Muller code)

```

phy_h      : 12*SM + 7*h + 3*reset + 12*t + 12*tdg
phy_sdg    : 7*s + 7*t
phy_s      : 7*sdg + 7*tdg
phy_cx     : 12*SM + 7*cx + 7*measure + 11*reset + 53*t + 53*tdg
phy_x      : 7*EC + 7*x
phy_z      : 7*EC + 3*t + 3*tdg + 7*z
phy_reset  : 6*SM + 7*reset + 24*t + 24*tdg
phy_t      : 8*t + 7*tdg
phy_tdg    : 7*t + 8*tdg
phy_cz     : 12*SM + 3*t + 3*tdg
phy_measure : 6*SM + measure + 2*t + 2*tdg

```

Here, SM refers to the syndrome measurement gadget and EC refers to the error correction gadget. As convention, we will consider the case where $\text{SM} = \text{EC}$, and the syndrome measurement frequency is after each gate, thus the value is the sum of all logical gates.

This series of embeddings exposes four free parameters: the eigenvalue precision ep , the number of qubits required for the Hamiltonian qu , the Trotter steps r , and the hardware's depolarizing noise depol . Interpreting eigenvalue

precision as binary energy precision, the least significant bit should not be larger than the target chemical accuracy, which is taken to be $1\text{kcal/mol} \approx 1.6 \times 10^{-3} Ha$, requiring about 10 precision bits. For the 28-qubit N_2H_4 Hamiltonian, taking the Trotter steps of 5 and a depolarizing error rate of 10^{-4} (optimistic by currently available QPUs), we obtain the estimate shown in Box 5.

Box 5: Full-stack resources for Steane code

num_qubits	: 1.1×10^3		
phy_h	: 1.6×10^{12}	phy_sdg	: 3.2×10^{11}
phy_cx	: 2.6×10^{12}	phy_x	: 5.7×10^{11}
phy_z	: 6.6×10^{11}	phy_reset	: 1.2×10^{12}
phy_t	: 2.5×10^{11}	phy_tdg	: 2.1×10^{11}
phy_cz	: 1.1×10^{12}	phy_measure	: 5.5×10^{11}

Extension to surface code resources

Next, we replace the Steane code with the surface code [109] and present the corresponding resource estimates in Box 6. The estimates are obtained using the implementation given in [108], with the parameters listed in Table 4. Note that the parameter `GridSynth accuracy` shown in the table corresponds to the optimal synthesis accuracy.

To assess scalability, we extend the study to a range of problem sizes. As illustrated in Figure 20, the optimal GridSynth accuracy achieves a consistent runtime reduction of approximately 85.7% for ϵ_p values ranging from 2 to 14.

Box 6: Resources for surface code (with optimal GridSynth accuracy)

runtime (s)	: 1.3×10^6	num_qubits	: 1.0×10^6
-------------	---------------------	------------	---------------------

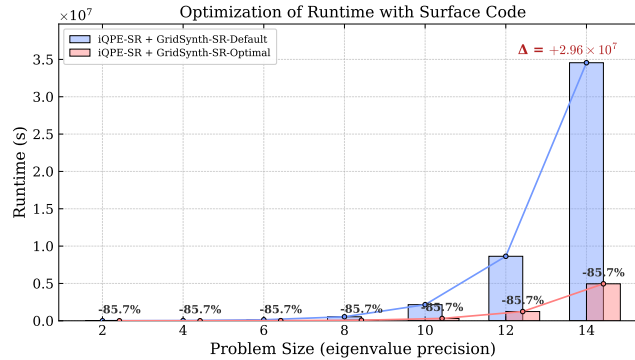


Figure 20 Surface-code runtime for the Hydrazine iQPE stack versus eigenvalue precision, comparing a fixed baseline GridSynth accuracy ($\epsilon = 10^{-10}$) against ϵ_{opt} . Using the surface-code parameters of Table 4, the optimal accuracy yields a nearly constant runtime reduction of $\approx 85.7\%$ across all problem sizes.

Extension to partial FTQC

The estimates above bracket two extremes of the fault-tolerance spectrum. Small codes, such as the Steane code with T-teleportation from the Reed-Muller code, are attractive in the early fault-tolerant regime for their low qubit overhead, whereas the surface code offers a scalable path to full fault tolerance at a substantially higher physical qubit cost. Partial fault tolerance on a topological patch bridges these regimes. In particular, the STAR architecture [88] error-corrects the Clifford gates while implementing rotations as space-time efficient analog operations, and its recent STAR ver. 3 refinement [110] introduces a STAR-magic mutation protocol that realizes analog rotation gates within a single surface code patch using a Clifford+ $T+\phi$ gate set. Recent application-level studies further motivate this regime, with STAR-based compilation of Trotterized time evolution enabling ground-state energy estimation of the 2D Hubbard model [111] and chemically accurate quantum phase estimation [34]. As a future extension, the presented logical stack model can be further composed with the resource estimates of STAR ver. 3, adding it as an alternative error-correction adapter alongside the Steane and

surface code layers, so that the same iQPE and decomposition models feed directly into a partially fault-tolerant estimate.

Future work

This case study can be extended to larger molecules, other EFT QPE algorithms, alternative universal QEC schemes, and decomposition strategies. With this study, we highlight the utility of layer-specific models coalescing to form a composite digital twin of the EFT stack, enabling computationally tractable resource estimation with minimal expert involvement.

4.3 Scenario III: Parameterized circuits and hardware constraints for optimization

Summary

- **Novelties used:** Full-stack flexibility ($\mathcal{N}1$), library support ($\mathcal{N}2$), and automated optimization ($\mathcal{N}4$).
- **Co-design variables:** Mixing ansatz, qubit connectivity and routing strategy, backend gate set, and zero-noise extrapolation (ZNE).
- **Key insight:** Reveals how sensitive performance is to ansatz and routing choices under noisy backend.

Background

Parameterized quantum circuit (PQC) based variational algorithms underpin many NISQ-era algorithms due to their simplicity, ability to use shallow noisy circuits, and wide applicability across domains, such as optimization, machine learning, finance, and drug discovery [112, 113, 114, 115, 116, 117]. However, their performance is highly sensitive to parameterized ansatz choices, noise characteristics, and qubit connectivity. Hence, recent papers have explored the importance of resource estimation and appropriate design choices in variational algorithms [118, 119].

Here we study the quantum approximate optimization algorithm, or rather its generalization, the quantum alternating operator ansätze (QAOA) applied to MaxCut, a standard NP-complete combinatorial optimization problem. QAOA can be tailored to quadratic unconstrained binary optimization (QUBO) problems, with extensions to higher-order and constrained optimization problems [120, 121]. Given its wide applicability, QAOA-based MaxCut presents a motivated benchmark for studying NISQ-era quantum resource estimation.

Most prior research implementing QAOA assumes idealized all-to-all connectivity, which is generally impractical in near-term devices. In this scenario, we systematically introduce constraints, such as connectivity and qubit routing, backend gates, and noisy hardware. We incorporate zero-noise extrapolation (ZNE) via linear and Richardson schemes and evaluate performance across noisy and mitigated settings. AutoQuREO’s generalized layer and hyperparameter abstractions, as described in Section 3.2, enable these heterogeneous variables to coexist within the optimization framework.

By jointly exploring hyperparameter ranges, AutoQuREO reveals trade-offs among expressivity, routing overhead, and mitigation efficacy. These results generalize naturally to more complex QML workloads as additional data and models are incorporated.

Setup

The following stages define the resource estimation case study:

1. **MaxCut problem:** Given a graph G , with N nodes and E edges, MaxCut aims to divide the nodes into sets N_1 and N_2 that maximize the number of edges connecting these two sets. Figure 21 shows the graph considered in this task.

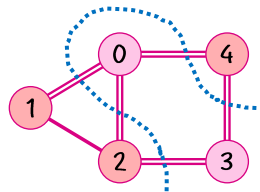


Figure 21 Input graph for MaxCut problem, with a chosen node ordering (the solution is independent of the ordering choice). For this simple example, the solution can be found by brute-force search, with the blue dotted line dividing the graph into two node sets: $\{0, 3\}$ and $\{1, 2, 4\}$.

2. **QAOA circuit** : Using (N, E) , we can construct a cost Hamiltonian H_C , as follows,

$$H_C = \sum_{(i,j) \in E} W_{ij} Z_i Z_j, \quad (22)$$

where W_{ij} is the weight of edge (i, j) , chosen here to be all 1. Minimization of H_C corresponds to the solution of the MaxCut problem. The QAOA is composed of alternating layers of trainable H_C and mixer Hamiltonian H_M ,

$$U(\alpha, \beta) = e^{-iH_C \alpha_1} e^{-iH_M \beta_1} \dots e^{-iH_C \alpha_p} e^{-iH_M \beta_p}, \quad (23)$$

repeated for p layers. $\alpha, \beta \in \mathbb{R}^p$ correspond to vectors of angles defining the parameters for the evolution duration. Several choices of the mixer Hamiltonians have been proposed in literature [122]. In this work, we focus on the following:

- **Vanilla-X**: composed of the sum of X operators over each qubit corresponding to the trainable mixer unitary layer $e^{-i\beta \sum_j X_j}$
- **Vanilla-Y**: composed of the sum of Y operators over each qubit corresponding to the trainable mixer unitary layer $e^{-i\beta \sum_j Y_j}$
- **Multiangle-X**: similar to Vanilla-X, with independent trainable parameters for each qubit, corresponding to a trainable mixer unitary $e^{-i \sum_j \beta_j X_j}$
- **Grover**: inspired by the Grover Search quantum algorithm, the ansatz implements a Grover-like selective phase shift mixing operators combined with a trainable parameter. The exact form of the operator depends on which solutions are feasible among the search space. In our example, all solutions are feasible resulting in a mixer unitary of the form $e^{-i\beta |s\rangle \langle s|}$, where $|s\rangle = |+\rangle^{\otimes n}$.
- **Digitized Counter Adiabatic (DCA)**: inspired by adiabatic evolution, DCA adds further trainable counter-adiabatic driving terms, obtained through a nested commutator approach of the adiabatic gauge potential [123], to the standard ansätze. The resulting ansätze is of the form,

$$U(\alpha, \beta, \gamma) = e^{-iH_C \alpha_1} e^{-iH_M \beta_1} e^{-iH_{DCA} \gamma_1} \dots e^{-iH_C \alpha_p} e^{-iH_M \beta_p} e^{-iH_{DCA} \gamma_p}, \quad (24)$$

The specific case we consider here is with $H_M = \sum_i X_i$, and refer to the ansatz as DCA-X. The number of layers p is also an important parameter; in general, larger layers result in higher solution accuracy. Here we choose 1 and 3 layers for our analysis.

3. **Routing**: In hardware backends, the quantum circuit needs to be routed depending on the connectivity and allowed backend gates. Here we focus on two types of hardware whose connectivity is shown in Figure 22,
- **Trapped-ion backend**: Allows entangling gates between all pairs of qubits, and very high fidelity single-qubit gates, albeit with slower gate-speed (of the order of μs), as listed in Table 5.
 - **Superconducting backend**: Allows entangling gates only between neighboring qubits, with much higher gate-speed (of the order of ns), as listed in Table 5.

For both backends, we use the SABRE routing method provided by Qiskit [72].

Table 5 Runtime and fidelities of quantum gates, measurements and T_1, T_2 for superconducting and trapped-ion backend obtained from recent literature.

	Superconducting		Trapped-ion	
	Gate time	Fidelity	Gate time	Fidelity
1-qubit rotation	8ns	99.997% [124]	13 μs	99.99998% [125]
CX gate	60ns	99.96% [126]	60 μs	99.97% [127]
Measurement	-	99.9% [128]	-	99.9993% [129]
T_1	50 – 300 μs	-	1 – 10s	-
T_2	50 – 200 μs	-	1 – 10s	-

4. **Circuit optimization**: The routed quantum circuits are further simplified by using Qiskit's built-in circuit optimization routine. The level of optimization ranges from 0 to 3, with level 0 performing no optimization and just the minimal amount of work to make the circuit runnable on the selected backend, and level 3

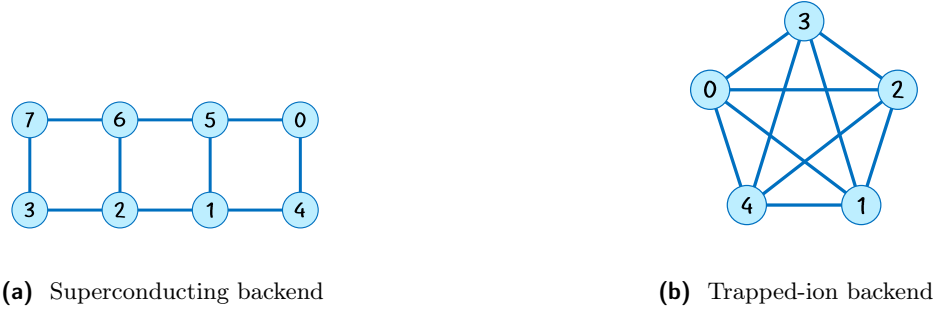


Figure 22 Backend connectivities. (a) Superconducting backend with grid connectivity among 4×2 qubits, and (b) Trapped-ion backend with all-to-all connectivity among 5 qubits are considered in this scenario. Note: mapping is agnostic to the arbitrary physical qubit IDs. The gate-times, fidelity values, measurement error and T_1, T_2 values are mentioned in Table 5. We use the same time and fidelity values for all qubits in a backend, and the T_1, T_2 values are randomly sampled for each qubit from the range of experimentally obtainable values.

spending the most amount of effort (and typically runtime) to try to optimize the circuit by combining multiple gates into more efficient operations. Here, the optimization level is set to 3.

5. **Error mitigation:** In NISQ devices, error-mitigation is an important part of obtaining useful information from quantum measurements. Here, we use the ZNE technique [66], which runs the circuit at various noise levels by increasing the circuit depth and extrapolates to the zero-noise measurement outcome. We compare the bare noisy circuit against ZNE using linear extrapolation with global circuit folding, and Richardson extrapolation with random gate folding. We use the ZNE modules from the Mitiq library [130].
6. **Training preparation:** To train the routed and optimized parametrized circuit, we need a classical optimizer. Here we choose the gradient-free COBYLA optimizer [131] with maximum iterations 1000 and tolerance 10^{-6} . For each evaluation, we set the number of measurement shots to be 10000.
7. **Training execution:** This layer simply runs the final training pipeline based on all the configurations set in the above layers.
8. **MaxCut evaluation:** The solution is evaluated by measuring the approximation ratio defined as,

$$\text{Approximation Ratio (AR)} = \frac{\langle \psi(\theta^*) | \hat{H}_C | \psi(\theta^*) \rangle}{\min_eigval(H_C)} \quad (25)$$

where $|\psi(\theta^*)\rangle$ is the optimized ansätze and $\min_eigval(H_C)$ is the minimum eigenvalue of H_C . The correct solution corresponds to $AR = 1$, with higher values corresponding to better accuracy of the solution.

Given these layers and corresponding hyperparameter choices, our aim is to identify how these choices affect the final approximation ratio and the resources required. The hyperparameter choices and resources are summarized in Table 6.

Results

For each backend, we plot the AR and runtime for various error-mitigation strategies, ansätze choice and number of layers in Figures 23 and 24. To remove biases introduced by parameter initialization, we run 5 independent trials for each configuration and choose the best AR as the outcome for that configuration. Each ansätze choice is marked in separate colors, with solid color for 3 layers and patterned for 1 layer, and the error-mitigation strategies shown on the x-axis. Using Figure 23, a few insightful trends can be identified:

- Most prominently, multiangle-X ansätze consistently outperforms all other ansätze choices which can be attributed to the additional trainable parameters. On the other hand, Grover ansätze shows very low AR for all configurations.
- Increasing the number of layers has improved AR for all ansätze choices except Vanilla-Y, suggesting that its maximum possible accuracy might be limited for this problem.
- The impact of error mitigation on the outcome is dependent on other factors. The change is most noticeable for Vanilla-X and DCA-X. Vanilla-X with a superconducting backend shows similar performance with and

Table 6 QRE layers and associated hyperparameter choices considered in this scenario. The optimization is done to minimize runtime and maximize the Approximation Ratio.

Layers	Hyperparameters	Ranges
MaxCut problem	Graph	
QAOA circuit	QAOA mixer	[Vanilla-X, Multiangle-X, DCA-X, Vanilla-Y, Grover]
	Layers	[1, 3]
Routing	Backend	[Trapped Ion all-to-all, Superconducting grid]
	Method	[SABRE]
Circuit optimization	Optimization level	[3]
Error mitigation	Strategy	[None, ZNE Linear, ZNE Richardson]
Training preparation	Tolerance	[1e-6]
	Max iterations	[1000]
	Optimizer	[COBYLA]
	Shots	[10000]
Train execution	-	
MaxCut evaluation	-	

Resources
Number of qubits
Approximation ratio
Runtime

without error mitigation, with a slight dip in AR for the ZNE linear scheme. On the other hand, for the trapped-ion backend, the ZNE linear scheme shows a significant dip, whereas the ZNE Richardson scheme slightly improves the AR. The trend for DCA-X is more clearly visible. For superconducting hardware, performance degrades as we move from bare circuits to the ZNE Richardson scheme, whereas the opposite is true for the trapped-ion backend. The circuits in DCA-X are deeper than the other ansätze choices except Grover, which explains the degradation in performance when using error mitigation on superconducting hardware with short coherence timescales ($\sim \mu s$) compared to increasing AR on trapped ion, which can support much longer coherence times ($\sim s$).

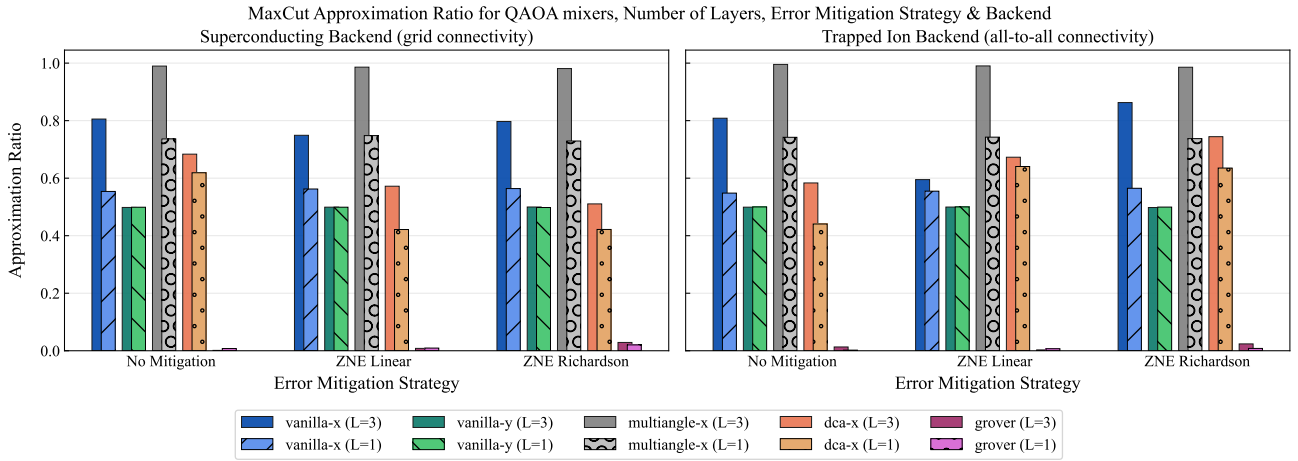


Figure 23 Comparison of Approximation Ratio for the MaxCut problem using different ansätze choices (marked in different colors), number of layers (marked in solid and patterns), backend and error-mitigation strategies in the QRE stack.

A similar comparison of their total runtime shows that multiangle-X, DCA-X, and Grover require much larger total runtime, compared to other ansätze choices. Since multiangle-X has many more trainable parameters, it requires many more function evaluations in the classical optimizer COBYLA, resulting in a larger runtime despite having similar depth to vanilla-X and vanilla-Y. Grover and DCA-X both require deeper circuits by design, and DCA-X also contains additional trainable parameters. Due to much slower gate-times, the trapped ion platform requires $\sim 10^3 - 10^4$ times longer. Additionally, the error-mitigation strategies add an order of magnitude of overhead to the total runtime in all cases.

To quantify the impact of various hyperparameter choices on the AR, we use the η^2 effect size, which measures the contribution of each hyperparameter to the AR. Unlike p -value, which gives a binary decision if the correlation

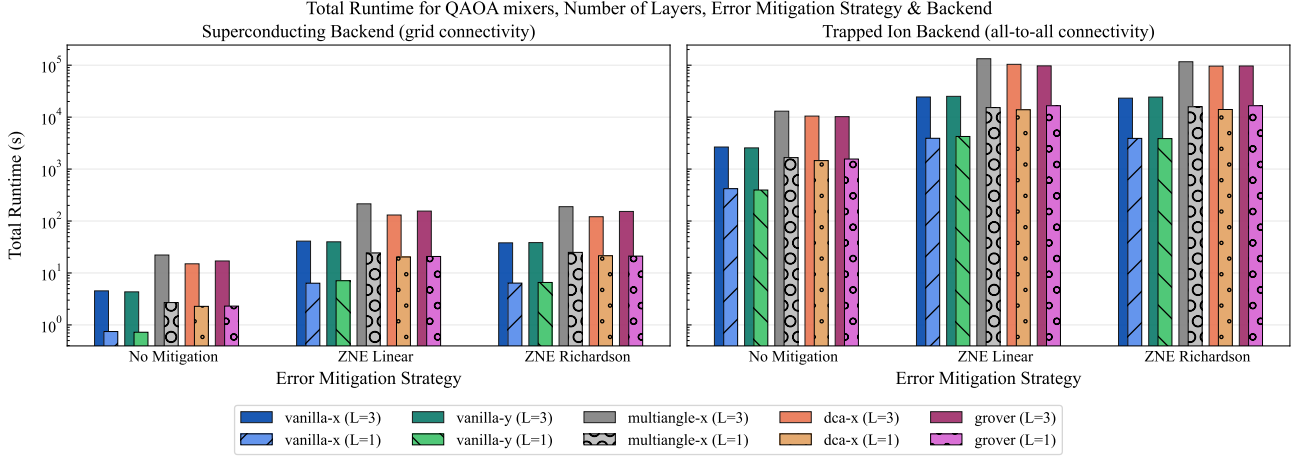


Figure 24 Comparison of total runtime for the MaxCut problem using different ansätze choices (marked in different colors), number of layers (marked in solid and patterns), backend, and error-mitigation strategies in the QRE stack.

between two variables is statistically significant, η^2 measures the size of the correlation. The largest impact is due to the QAOA type reflected in its η^2 value of 0.9073. The next significant contributing factor is the number of layers with an η^2 value of 0.0357, as more layers have generally demonstrated improved AR values. The contribution from error-mitigation and backend, on the other hand, has been mixed and dependent on specific configuration choices. Thus, for larger instances, where conducting extensive DSE becomes intractable, the user can focus their efforts on tuning the design choices with higher sensitivity.

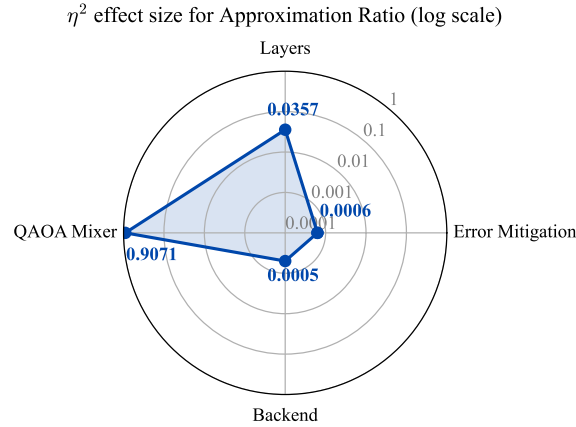


Figure 25 η^2 effect size quantifying the impact of each hyperparameter on Approximation Ratio. $\eta^2 \sim 0.01$ corresponds to a small effect, $\eta^2 \sim 0.06$ corresponds to a medium effect and $\eta^2 \sim 0.14$ higher values correspond to a large effect.

Using AutoQuREO, we can analyze the impact of various hyperparameters and backends on AR and runtime within a single framework, and identify the best configurations that maximize AR while minimizing runtime. For heuristic algorithms like QAOA, such analysis is especially helpful for understanding which hyperparameter choices to make as the number of qubits increases to obtain reasonable answers on noisy hardware. Due to its flexibility, AutoQuREO can be used to study a much broader range of variational algorithms [132, 133, 134] and to analyze useful trends [135, 136] to predict optimal choices under realistic hardware constraints. This transition from idealized simulation assumptions to realistic hardware-aware digital twins is precisely the intended use case of AutoQuREO workflow introduced in Section 3.9.

5 Discussion

This research presents the AutoQuREO quantum resource estimation (QRE) framework. AutoQuREO is designed with a design philosophy to democratize full-stack QRE across heterogeneous layers of the quantum computing stack, spanning algorithms, compilation, error correction, and hardware backends. Rather than constraining

users to a fixed abstraction (e.g., FTQC-centric pipelines), the framework exposes layer-wise modularity, allowing researchers, for example, to focus on a target layer while leveraging standardized tooling for the remaining stack for co-design and optimization. This is enabled through a reusable library of codes and surrogate models, reducing the overhead of cross-domain expertise. AutoQuREO bridges estimation and optimization by embedding multi-objective design space exploration (DSE) directly into the workflow, enabling not only prediction of resource metrics but also selection of optimal configurations for hardware deployment. The framework enables flexibility in analysis, allowing users to arbitrarily visualize trade-offs across hyperparameters and resources. A key distinguishing feature is the integration of neuro-symbolic (NeSy) explainable AI (XAI) resource surrogate modeling. This integrates the advantages from expressive learning (e.g., neural networks, neuro-evolution) with interpretable symbolic regression, enabling scalability across regimes while remaining amenable to analytical reasoning and formalization.

Through the representative case studies, we demonstrate AutoQuREO’s ability to uncover non-trivial insights in multi-layer quantum solutions. The scenario-driven methodology enables users to formalize questions of interest, such as optimizing a target resource while constraining others, and systematically explore the resulting trade-offs towards realistic deployment.

Looking forward, several backend and systems-level extensions can further enhance the applicability of AutoQuREO. Integration with emerging intermediate representations such as Bloqs, QREF, QIR [137], and MLIR [138, 139, 140, 141], can provide interoperability across quantum software ecosystems. Additionally, GPU-accelerated QRE and model training can significantly accelerate surrogate synthesis and large-scale DSE. Another promising direction of interest is compiler phase ordering [142, 143], treating compilation passes as tunable parameters that influence overall resource allocation. Such extensions would further blur the boundary between quantum compilation in deployment pipelines and QRE, enabling holistic optimization of quantum middleware.

AutoQuREO can benefit from advanced automation methods, such as active learning [51], to achieve surrogate modeling with less data. Static analysis techniques can complement compiler-driven QRE by providing formal constraints and invariants. Enhancements to symbolic regression, including the incorporation of reusable gadgets [144], can improve compositionality and interpretability of learned models. Furthermore, large language models (LLMs) offer opportunities for autoformalization [145, 146, 147] of code-to-model pipelines, while the adapter abstraction can be extended as a model context protocol (MCP) [148], enabling seamless interoperability between heterogeneous modeling agents.

In the broader context of AutoQC [149], AutoQuREO contributes a critical component toward quantifying quantum advantage in an automated quantum solution workflow. A natural extension is the integration of AutoQuREO with quantum algorithm design automation (QADA), in which resource estimation and algorithm synthesis interact in a closed loop.

Finally, we envision AutoQuREO as a community-driven platform for advancing full-stack quantum computing research. The extensible library of layers, models, adapters, and scenarios is designed to facilitate contributions from researchers across domains, fostering a shared ecosystem of reusable components and benchmarks.

Acknowledgements

The authors thank Kushagra Garg for insightful discussions regarding error composability; and Muhilan Murugesan and Tania Sidana for discussions on correction schemes for modeling. We thank Yasuhiro Endo, Hirotaka Oshima, Shinji Kikuchi, Shintaro Sato, and Vivek Mahajan for valuable project guidance and comments on the article.

The authors used AI-assisted editing tools to improve grammar and clarity during manuscript preparation. The manuscript has been carefully reviewed by the authors, and they take full responsibility for the final content.

Author contributions

A.S. and H.O. conducted the requirement analysis to identify gaps in current methods, conceptualized the distinguishing novelties, designed the software workflow and led the AutoQuREO implementation. S.N.A. integrated PySR with NEAT and Optuna for surrogate synthesis and conducted the LER-PER results presented in § 4.2.1. A.P. conducted the co-design scenario in § 4.1, H.O. and A.S. conducted the co-design scenario in § 4.2, and R.B. conducted the co-design scenario in § 4.3. P.P.K. conducted the model benchmarking studies in § 4.2.3. K.S. was instrumental in initiating the project, securing necessary resources and overseeing project road map. All authors participated in the review and editing of the manuscript.

References

- [1] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 11–20, 1993.
- [2] Amara Katabarwa, Katerina Gratsea, Athena Caesura, and Peter D Johnson. Early fault-tolerant quantum computing. *PRX quantum*, 5(2):020101, 2024.
- [3] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando GSL Brandao, David A Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019.
- [4] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, 2025.
- [5] Koen Bertels, Aritra Sarkar, Thomas Hubregtsen, M Serrao, Abid A Mouedenne, Amitabh Yadav, A Krol, Imran Ashraf, and C Garcia Almudever. Quantum computer architecture toward full-stack quantum accelerators. *IEEE Transactions on Quantum Engineering*, 1:1–17, 2020.
- [6] Yunong Shi, Pranav Gokhale, Prakash Murali, Jonathan M Baker, Casey Duckering, Yongshan Ding, Natalie C Brown, Christopher Chamberland, Ali Javadi-Abhari, Andrew W Cross, et al. Resource-efficient quantum computing by breaking abstractions. *Proceedings of the IEEE*, 108(8):1353–1370, 2020.
- [7] Colin Campbell, Frederic T Chong, Denny Dahl, Paige Frederick, Palash Goiporia, Pranav Gokhale, Benjamin Hall, Salahdeen Issa, Eric Jones, Stephanie Lee, et al. Superstaq: Deep optimization of quantum programs. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1020–1032. IEEE, 2023.
- [8] Katerina Gratsea and Matthew Otten. Achieving utility-scale applications through full stack co-design of fault tolerant quantum computers. *arXiv preprint arXiv:2510.26547*, 2025.
- [9] Ryan Babbush, Robbie King, Sergio Boixo, William Huggins, Tanuj Khattar, Guang Hao Low, Jarrod R McClean, Thomas O’Brien, and Nicholas C Rubin. The grand challenge of quantum applications. *arXiv preprint arXiv:2511.09124*, 2025.
- [10] Nils Quetschlich, Mathias Soeken, Prakash Murali, and Robert Wille. Utilizing resource estimation for the development of quantum computing applications. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 232–238. IEEE, 2024.
- [11] Michael E Beverland, Prakash Murali, Matthias Troyer, Krysta M Svore, Torsten Hoefler, Vadym Kliuchnikov, Guang Hao Low, Mathias Soeken, Aarthi Sundaram, and Alexander Vaschillo. Assessing requirements to scale to practical quantum advantage. *arXiv preprint arXiv:2211.07629*, 2022.
- [12] Wim van Dam, Mariia Mykhailova, and Mathias Soeken. Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation. In *Proceedings of the SC’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 1414–1419, 2023.
- [13] Matthew P Harrigan, Tanuj Khattar, Charles Yuan, Anurudh Peduri, Noureldin Yosri, Fionn D Malone, Ryan Babbush, and Nicholas C Rubin. Expressing and analyzing quantum algorithms with qualtran. *arXiv preprint arXiv:2409.04643*, 2024.
- [14] PsiQ. Psiq/bartiq: Bartiq. Accessed: 2026-02-23.
- [15] SN Saadatmand, Tyler L Wilson, Mark J Hodson, Mark Field, Simon J Devitt, Madhav Krishnan Vijayan, Alan Robertson, Thinh P Le, Jannis Ruh, Alexandru Paler, et al. Superconducting qubits at the utility scale: The potential and limitations of modularity. *arXiv preprint arXiv:2406.06015*, 2024.
- [16] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M Sohaib Alam, Guillermo Alonso-Linaje, Bharath AkashNarayanan, Ali Asadi, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018.
- [17] Colin Campbell, Rich Rines, Victory Omole, Tina Oberoi, Palash Goiporia, Rayat Roy, R Peyton Cline, Eric B Jones, and Teague Tomesh. Resource estimation via efficient compilation of key quantum primitives. *arXiv preprint arXiv:2604.01376*, 2026.
- [18] Jan Krzyszkowski and Marcin Niemiec. Analysis of surface code algorithms on quantum hardware using the qrisp framework. *Electronics*, 14(23):4707, 2025.
- [19] Masoud Mohseni, Artur Scherer, K Grace Johnson, Oded Wertheim, Matthew Otten, Navid Anjum Aadit, Yuri Alexeev, Kirk M Bresniker, Kerem Y Camsari, Barbara Chapman, et al. How to build a quantum supercomputer: Scaling from hundreds to millions of qubits. *arXiv preprint arXiv:2411.10406*, 2024.
- [20] David Elieser Deutsch. Quantum computational networks. *Proceedings of the royal society of London. A. mathematical and physical sciences*, 425(1868):73–90, 1989.
- [21] Sibasish Mishra, Aritra Sarkar, and Sebastian Feld. EQISA: Energy-efficient quantum instruction set architecture using sparse dictionary learning. *arXiv preprint arXiv:2603.20646*, 2026.
- [22] Torsten Hoefler, Thomas Häner, and Matthias Troyer. Disentangling hype from practicality: On realistically achieving quantum advantage. *Communications of the ACM*, 66(5):82–87, 2023.
- [23] Emmanouil Giortamis, Francisco Romão, Nathaniel Tornow, and Pramod Bhatotia. {QOS}: Quantum operating system. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 429–447, 2025.

- [24] Yuval R Sanders, Dominic W Berry, Pedro CS Costa, Louis W Tessler, Nathan Wiebe, Craig Gidney, Hartmut Neven, and Ryan Babbush. Compilation of fault-tolerant quantum heuristics for combinatorial optimization. *PRX quantum*, 1(2):020312, 2020.
- [25] Mohsen Bagherimehrab, Yuval R Sanders, Dominic W Berry, Gavin K Brennen, and Barry C Sanders. Nearly optimal quantum algorithm for generating the ground state of a free quantum field theory. *PRX Quantum*, 3(2):020364, 2022.
- [26] Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits. *arXiv preprint arXiv:2505.15917*, 2025.
- [27] Junyu Fan, Matthew Steinberg, Alexander Jahn, Chunjun Cao, Aritra Sarkar, and Sebastian Feld. Lego_hqec: Automating the analysis, construction, and decoding of holographic quantum codes. *arXiv preprint arXiv:2410.22861*, 2024.
- [28] Martin Suchara, John Kubiawicz, Arvin Faruque, Frederic T Chong, Ching-Yi Lai, and Gerardo Paz. Qure: The quantum resource estimator toolbox. In *2013 IEEE 31st international conference on computer design (ICCD)*, pages 419–426. IEEE, 2013.
- [29] Kevin Obenland, Justin Elenewski, Arthur Kurlej, Joe Belarge, John Blue, and Robert Rood. pyliqtr (lincoln laboratory quantum algorithm test and research), 2023.
- [30] SN Saadatmand, Tyler L Wilson, Mark Field, Madhav Krishnan Vijayan, Thinh P Le, Jannis Ruh, Arshpreet Singh Maan, Ioana Mofic, Athena Caesura, Alexandru Paler, et al. Fault-tolerant resource estimation using graph-state compilation on a modular superconducting architecture. *arXiv preprint arXiv:2406.06015*, 2024.
- [31] Zapata AI. benchq: Resource estimation for fault-tolerant quantum computation., 2023.
- [32] Greg Bowen, Athena Caesura, Simon Devitt, and Madhav Krishnan Vijayan. Design and efficiency in graph-state computation. *arXiv preprint arXiv:2502.18985*, 2025.
- [33] Riki Toshio, Yutaro Akahoshi, Jun Fujisaki, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii. Practical quantum advantage on partially fault-tolerant quantum computer. *Physical Review X*, 15(2):021057, 2025.
- [34] Shota Kanasugi, Riki Toshio, Kazunori Maruyama, and Hirotaka Oshima. Enabling chemically accurate quantum phase estimation in the early fault-tolerant regime. *arXiv preprint arXiv:2603.22778*, 2026.
- [35] Bonan Su, Yuan Feng, Li Zhou, and Mingsheng Ying. Resource estimation for fault-tolerant quantum programs. *arXiv preprint arXiv:2608.04573*, 2026.
- [36] Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T Chong, and Margaret Martonosi. Scaffcc: Scalable compilation and analysis of quantum programs. *Parallel Computing*, 45:2–17, 2015.
- [37] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S Bishop, Steven Heidel, Colm A Ryan, Prasahnt Sivarajah, John Smolin, Jay M Gambetta, et al. Openqasm 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3):1–50, 2022.
- [38] Susan L Graham, Peter B Kessler, and Marshall K McKusick. Gprof: A call graph execution profiler. *ACM Sigplan Notices*, 17(6):120–126, 1982.
- [39] Adrien Suau, Gabriel Staffelbach, and Aida Todri-Sanial. Qprof: A gprof-inspired quantum profiler. *ACM Transactions on Quantum Computing*, 4(1):1–28, 2022.
- [40] Felix Zilk, Alessandro Tundo, Vincenzo De Maio, and Ivona Brandic. Breaking down quantum compilation: Profiling and identifying costly passes. *arXiv preprint arXiv:2504.15141*, 2025.
- [41] Long Pham, Feras A Saad, and Jan Hoffmann. Robust resource bounds with static analysis and bayesian inference. *Proceedings of the ACM on Programming Languages*, 8(PLDI):76–101, 2024.
- [42] Long Pham, Yue Niu, Nathan Glover, Feras Saad, and Jan Hoffmann. Integrating resource analyses via resource decomposition. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3811–3840, 2025.
- [43] Andrea Colledan and Ugo Dal Lago. Flexible type-based resource estimation in quantum circuit description languages. *Proceedings of the ACM on Programming Languages*, 9(POPL):1386–1416, 2025.
- [44] Giulia Meuli, Mathias Soeken, Martin Roetteler, and Thomas Häner. Enabling accuracy-aware quantum compilers using symbolic resource estimation. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–26, 2020.
- [45] Dmitrijs Zaparanuks and Matthias Hauswirth. Algorithmic profiling. In *Proceedings of the 33rd ACM SIGPLAN conference on Programming Language Design and Implementation*, pages 67–76, 2012.
- [46] Catherine McGeoch, Peter Sanders, Rudolf Fleischer, Paul R Cohen, and Doina Precup. Using finite experiments to study asymptotic performance. In *Experimental algorithmics: from algorithm design to robust and efficient software*, pages 93–126. Springer, 2002.
- [47] Henry Gordon Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society*, 74(2):358–366, 1953.
- [48] Abhishek Purohit, Maninder Kaur, Zeki Can Seskir, Matthew T Posner, and Araceli Venegas-Gomez. Building a quantum-ready ecosystem. *IET Quantum Communication*, 5(1):1–18, 2024.
- [49] Erich Gamma. *Design patterns: elements of reusable object-oriented software*. Pearson Education India, 1995.
- [50] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. t| ket>: a retargetable compiler for nisq devices. *Quantum Science & Technology*, 6(1):014003, 2021.
- [51] Prasanna Balaprakash, Robert B Gramacy, and Stefan M Wild. Active-learning-based surrogate models for empirical performance tuning. In *2013 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 1–8. IEEE, 2013.

- [52] H Magalhães, F Marques, B Liu, João Pombo, P Flores, Jorge Ambrósio, and S Bruni. An optimization approach to generate accurate and efficient lookup tables for engineering applications. In *International Conference on Engineering Optimization*, pages 1446–1457. Springer, 2018.
- [53] Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression. jl. *arXiv preprint arXiv:2305.01582*, 2023.
- [54] Kenneth O Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary computation*, 10(2):99–127, 2002.
- [55] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [56] Mark B Ring. Child: A first step towards continual learning. *Machine Learning*, 28(1):77–104, 1997.
- [57] Norihiro Kakuko, Shun Gokita, Naoyuki Masumoto, Keita Matsumoto, Kosuke Miyaji, Takafumi Miyanaga, Toshio Mori, Haruki Nakayama, Keita Sasada, Yasuhito Takamiya, et al. A practical open-source software stack for a cloud-based quantum computing system. *arXiv preprint arXiv:2507.23165*, 2025.
- [58] Aaron Meurer, Christopher P Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K Moore, Sartaj Singh, et al. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, 2017.
- [59] Madhav R Muthyala, Farshud Sorourifar, You Peng, and Joel A Paulson. Symantic: An efficient symbolic regression method for interpretable and parsimonious model discovery in science and beyond. *Industrial & Engineering Chemistry Research*, 64(6):3354–3369, 2025.
- [60] Madhav Muthyala, Farshud Sorourifar, and Joel A Paulson. Torchsisso: a pytorch-based implementation of the sure independence screening and sparsifying operator for efficient and interpretable model discovery. *Digital Chemical Engineering*, 13:100198, 2024.
- [61] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. Mqt bench: Benchmarking software and design automation tools for quantum computing. *Quantum*, 7:1062, 2023.
- [62] Jacobo Padín Martínez, Vicente P Soloviev, Alejandro Borrallo Rentero, Antón Rodríguez Otero, Raquel Alfonso Rodríguez, and Michal Krompiec. Pauli correlation encoding for budget-constraint optimization. *arXiv preprint arXiv:2602.17479*, 2026.
- [63] David Kremer, Victor Villar, Hanhee Paik, Ivan Duran, Ismael Faro, and Juan Cruz-Benito. Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning. *arXiv preprint arXiv:2405.13196*, 2024.
- [64] Andrew J Daley, Immanuel Bloch, Christian Kokail, Stuart Flannigan, Natalie Pearson, Matthias Troyer, and Peter Zoller. Practical quantum advantage in quantum simulation. *Nature*, 607(7920):667–676, 2022.
- [65] Naomichi Hatano and Masuo Suzuki. Finding exponential product formulas of higher orders. In *Quantum annealing and other optimization methods*, pages 37–68. Springer, 2005.
- [66] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J Zeng. Digital zero noise extrapolation for quantum error mitigation. In *2020 IEEE international conference on quantum computing and engineering (QCE)*, pages 306–316. IEEE, 2020.
- [67] AA Avtandilyan and WV Pogosov. Optimal-order trotter–suzuki decomposition for quantum simulation on noisy quantum computers: Aa avtandilyan, wv pogosov. *Quantum Information Processing*, 24(1):8, 2024.
- [68] Jue Xu, Chu Zhao, Junyu Fan, and Qi Zhao. Exponentially decaying quantum simulation error with noisy devices. *arXiv preprint arXiv:2504.10247*, 2025.
- [69] Avimita Chatterjee, Sonny Rappaport, Anish Giri, Sonika Johri, Timothy Proctor, David E Bernal Neira, Pratik Sathe, and Thomas Lubinski. A comprehensive cross-model framework for benchmarking the performance of quantum hamiltonian simulations. *IEEE Transactions on Quantum Engineering*, 2025.
- [70] Noah Siekierski, Stefan Seritan, Neer Patel, Siyuan Niu, Thomas Lubinski, and Timothy Proctor. Software for creating scalable benchmarks from quantum algorithms. *arXiv preprint arXiv:2511.02134*, 2025.
- [71] J Robert Johansson, Paul D Nation, and Franco Nori. Qutip: An open-source python framework for the dynamics of open quantum systems. *Computer physics communications*, 183(8):1760–1772, 2012.
- [72] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit, 2024.
- [73] Goran Lindblad. On the generators of quantum dynamical semigroups. *Communications in mathematical physics*, 48(2):119–130, 1976.
- [74] Giovanni Di Bartolomeo, Michele Vischi, Tommaso Feri, Angelo Bassi, and Sandro Donadi. Efficient quantum algorithm to simulate open systems through a single environmental qubit. *Physical Review Research*, 6(4):043321, 2024.
- [75] Gumaro Rendon, Jacob Watkins, and Nathan Wiebe. Improved accuracy for trotter simulations using chebyshev interpolation. *Quantum*, 8:1266, 2024.
- [76] Pegah Mohammadipour and Xiantao Li. Reducing circuit depth in lindblad simulation via step-size extrapolation. *arXiv preprint arXiv:2507.22341*, 2025.
- [77] Earl Campbell. A random compiler for fast hamiltonian simulation. *arXiv preprint arXiv:1811.08017*, 2018.
- [78] Andrew J Daley. Quantum trajectories and open many-body quantum systems. *Advances in Physics*, 63(2):77–149, 2014.

- [79] Yasunari Suzuki, Suguru Endo, Keisuke Fujii, and Yuuki Tokunaga. Quantum error mitigation as a universal error reduction technique: Applications from the nisq to the fault-tolerant quantum computing eras. *PRX quantum*, 3(1):010345, 2022.
- [80] Misty A Wahl, Andrea Mari, Nathan Shammah, William J Zeng, and Gokul Subramanian Ravi. Zero noise extrapolation on logical qubits by scaling the error correction code distance. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 888–897. IEEE, 2023.
- [81] Andrew Steane. Multiple-particle interference and quantum error correction. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 452(1954):2551–2577, 1996.
- [82] Lucas Daguerre and Isaac H Kim. Code switching revisited: Low-overhead magic state preparation using color codes. *Physical Review Research*, 7(2):023080, 2025.
- [83] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for nisq-era quantum devices. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, pages 1001–1014, 2019.
- [84] Henry Zou, Matthew Treinish, Kevin Hartman, Alexander Ivrii, and Jake Lishman. Lightsabre: A lightweight and enhanced sabre algorithm. *arXiv preprint arXiv:2409.08368*, 2024.
- [85] Lucas Daguerre, Robin Blume-Kohout, Natalie C Brown, David Hayes, and Isaac H Kim. Experimental demonstration of high-fidelity logical magic states from code switching. *Physical Review X*, 15(4):041008, 2025.
- [86] Craig Gidney, Noah Shutt, and Cody Jones. Magic state cultivation: growing t states as cheap as cnot gates. *arXiv preprint arXiv:2409.17595*, 2024.
- [87] Chris N Self, Marcello Benedetti, and David Amaro. Protecting expressive circuits with a quantum error detection code. *Nature Physics*, 20(2):219–224, 2024.
- [88] Yutaro Akahoshi, Kazunori Maruyama, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii. Partially fault-tolerant quantum computing architecture with error-corrected clifford gates and space-time efficient analog rotations. *PRX quantum*, 5(1):010337, 2024.
- [89] Daniel Bultrini, Samson Wang, Piotr Czarnik, Max Hunter Gordon, Marco Cerezo, Patrick J Coles, and Lukasz Cincio. The battle of clean and dirty qubits in the era of partial error correction. *Quantum*, 7:1060, 2023.
- [90] Edwin Tham and Nicolas Delfosse. Optimized clifford noise reduction: Theory, simulations and experiments. *Quantum*, 9:1829, 2025.
- [91] Daniel Litinski. A game of surface codes: Large-scale quantum computing with lattice surgery. *Quantum*, 3:128, 2019.
- [92] Nikolas P Breuckmann and Jens Niklas Eberhardt. Quantum low-density parity-check codes. *PRX quantum*, 2(4):040101, 2021.
- [93] Tianyi Hao, Amanda Xu, and Swamit Tannu. Reducing t gates with unitary synthesis. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1589–1604, 2026.
- [94] Neil J Ross and Peter Selinger. Optimal ancilla-free clifford+ t approximation of z-rotations. *Quantum Inf. Comput.*, 16(11&12):901–953, 2016.
- [95] Pau Escofet, Santiago Rodrigo, Artur Garcia-Sáez, Eduard Alarcón, Sergi Abadal, and Carmen G Almudéver. An accurate and efficient analytic model of fidelity under depolarizing noise oriented to large scale quantum system design. *Quantum Science and Technology*, 10(3):035061, 2025.
- [96] Daniel Litinski. Magic state distillation: Not as costly as you think. *Quantum*, 3:205, 2019.
- [97] Christopher M Dawson and Michael A Nielsen. The solovay-kitaev algorithm. *arXiv preprint quant-ph/0505030*, 2005.
- [98] Hayata Morisaki, Kaoru Sano, and Seiseki Akibue. Optimal ancilla-free clifford+ t synthesis for general single-qubit unitaries. *arXiv preprint arXiv:2510.05816*, 2025.
- [99] Vivek V Shende, Stephen S Bullock, and Igor L Markov. Synthesis of quantum logic circuits. In *Proceedings of the 2005 Asia and South Pacific Design Automation Conference*, pages 272–275, 2005.
- [100] Mathias Weiden, Ed Younis, Justin Kalloor, John Kubiawicz, and Costin Iancu. Improving quantum circuit synthesis with machine learning. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1–11. IEEE, 2023.
- [101] Korbinian Kottmann, David Wierichs, Guillermo Alonso-Linaje, and Nathan Killoran. Parameter-optimal unitary synthesis with flag decompositions. *arXiv preprint arXiv:2603.20376*, 2026.
- [102] Pablo Arnault, Pablo Arrighi, Steven Herbert, Evi Kasnetsi, and Tianyi Li. A typology of quantum algorithms. *arXiv preprint arXiv:2407.05178*, 2024.
- [103] Miroslav Dobšček, Göran Johansson, Vitaly Shumeiko, and Göran Wendin. Arbitrary accuracy iterative quantum phase estimation algorithm using a single ancillary qubit: A two-qubit benchmark. *Physical Review A—Atomic, Molecular, and Optical Physics*, 76(3):030306, 2007.
- [104] Alexandria J Moore, Yuchen Wang, Zixuan Hu, Sabre Kais, and Andrew M Weiner. Statistical approach to quantum phase estimation. *New Journal of Physics*, 23(11):113027, 2021.
- [105] Justin M Turney, Andrew C Simmonett, Robert M Parrish, Edward G Hohenstein, Francesco A Evangelista, Justin T Fermann, Benjamin J Mintz, Lori A Burns, Jeremiah J Wilke, Micah L Abrams, et al. Psi4: an open-source ab initio electronic structure program. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 2(4):556–565, 2012.

- [106] Anna M Krol and Zaid Al-Ars. Beyond quantum shannon decomposition: Circuit construction for n-qubit gates based on block-zxz decomposition. *Physical Review Applied*, 22(3):034019, 2024.
- [107] Anna M Krol, Aritra Sarkar, Imran Ashraf, Zaid Al-Ars, and Koen Bertels. Efficient decomposition of unitary matrices in quantum circuit compilers. *Applied Sciences*, 12(2):759, 2022.
- [108] Google. Qualtran documentation, 2023.
- [109] Craig Gidney and Austin G Fowler. Efficient magic state factories with a catalyzed $|CCZ\rangle$ to $2|T\rangle$ transformation. *Quantum*, 3:135, 2019.
- [110] Riki Toshio, Shota Kanasugi, Jun Fujisaki, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii. Star-magic mutation: Even more efficient analog rotation gates for early fault-tolerant quantum computer. *arXiv preprint arXiv:2603.22891*, 2026.
- [111] Yutaro Akahoshi, Riki Toshio, Jun Fujisaki, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii. Compilation of trotter-based time evolution for partially fault-tolerant quantum computing architecture. *PRX Quantum*, 6(4):040319, 2025.
- [112] Bela Bauer, Sergey Bravyi, Mario Motta, and Garnet Kin-Lic Chan. Quantum algorithms for quantum chemistry and quantum materials science. *Chemical Reviews*, 120(22):12685–12717, 2020.
- [113] Mario Motta and Julia E Rice. Emerging quantum computing algorithms for quantum chemistry. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 12(3):e1580, 2022.
- [114] Vojtěch Havlíček, Antonio D Córcoles, Kristan Temme, Aram W Harrow, Abhinav Kandala, Jerry M Chow, and Jay M Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, 2019.
- [115] Sergey Bravyi, David Gosset, Robert König, and Marco Tomamichel. Quantum advantage with noisy shallow circuits. *Nature Physics*, 16(10):1040–1045, 2020.
- [116] Diego Ristè, Marcus P Da Silva, Colm A Ryan, Andrew W Cross, Antonio D Córcoles, John A Smolin, Jay M Gambetta, Jerry M Chow, and Blake R Johnson. Demonstration of quantum advantage in machine learning. *npj Quantum Information*, 3(1):16, 2017.
- [117] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtshi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J Egger, Bruce G Elmegreen, et al. Challenges and opportunities in quantum optimization. *Nature Reviews Physics*, pages 1–18, 2024.
- [118] Tomáš Bezděk, Haomu Yuan, Vojtěch Novák, Silvie Illésová, and Martin Beseda. Classical optimization strategies for variational quantum algorithms: A systematic study of noise effects and parameter efficiency. *arXiv preprint arXiv:2511.09314*, 2025.
- [119] Ashish Kumar Patra, Vikas Dattatraya Ghevade, Ruchika Bhat, Rahul Maitra, et al. Resource estimation for vqe on small molecules: Impact of fermion mappings and hamiltonian reductions. *arXiv preprint arXiv:2512.01605*, 2025.
- [120] Leanto Sunny, Abhinav Rijal, and George Siopsis. Extending qaoa-gpt to higher-order quantum optimization problems. *arXiv preprint arXiv:2511.07391*, 2025.
- [121] Valter Uotila, Julia Ripatti, and Bo Zhao. Higher-order portfolio optimization with quantum approximate optimization algorithm. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 01–12. IEEE, 2025.
- [122] Kostas Blekos, Dean Brand, Andrea Ceschini, Chiao-Hui Chou, Rui-Hao Li, Komal Pandya, and Alessandro Summer. A review on quantum approximate optimization algorithm and its variants. *Physics Reports*, 1068:1–66, 2024.
- [123] Pieter W Claeyss, Mohit Pandey, Dries Sels, and Anatoli Polkovnikov. Floquet-engineering counterdiabatic protocols in quantum many-body systems. *Physical review letters*, 123(9):090602, 2019.
- [124] David A Rower, Leon Ding, Helin Zhang, Max Hays, Junyoung An, Patrick M Harrington, Ilan T Rosen, Jeffrey M Gertler, Thomas M Hazard, Bethany M Niedzielski, et al. Suppressing counter-rotating errors for fast single-qubit gates with fluxonium. *PRX Quantum*, 5(4):040342, 2024.
- [125] Molly C Smith, Aaron D Leu, Koichiro Miyanishi, Mario F Gely, and David M Lucas. Single-qubit gates with errors at the 10^{-7} level. *Physical Review Letters*, 134(23):230601, 2025.
- [126] Wei-Ju Lin, Hyunheung Cho, Yinqi Chen, Maxim G Vavilov, Chen Wang, and Vladimir E Manucharyan. 24 days-stable cnot gate on fluxonium qubits with over 99.9% fidelity. *PRX Quantum*, 6(1):010349, 2025.
- [127] CM Löschnauer, J Mosca Toba, AC Hughes, SA King, MA Weber, R Srinivas, R Matt, R Nourshargh, DTC Allcock, CJ Ballance, et al. Scalable, high-fidelity all-electronic control of trapped-ion qubits. *PRX Quantum*, 6(4):040313, 2025.
- [128] Can Wang, Feng-Ming Liu, He Chen, Yi-Fei Du, Chong Ying, Jian-Wen Wang, Yong-Heng Huo, Cheng-Zhi Peng, Xiaobo Zhu, Ming-Cheng Chen, et al. 99.9%-fidelity in measuring a superconducting qubit. *arXiv preprint arXiv:2412.13849*, 2024.
- [129] AS Sotirova, JD Leppard, A Vazquez-Brennan, SM Decoppet, F Pokorny, M Malinowski, and CJ Ballance. High-fidelity heralded quantum state preparation and measurement. *arXiv preprint arXiv:2409.05805*, 2024.
- [130] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter J. Karalekas, Andre A. Alves, Piotr Czarnik, Mohamed El Mandouh, Max H. Gordon, Yousef Hindy, Aaron Robertson, Purva Thakre, Misty Wahl, Danny Samuel, Rahul Mistri, Maxime Tremblay, Nick Gardner, Nathaniel T. Stemen, Nathan Shammah, and William J. Zeng. Mitiq: A software package for error mitigation on noisy quantum computers. *Quantum*, 6:774, Aug 2022.
- [131] Xavier Bonet-Monroig, Hao Wang, Diederick Vermetten, Bruno Senjean, Charles Moussa, Thomas Bäck, Vedran Dunjko, and Thomas E O’Brien. Performance comparison of optimization methods on variational quantum algorithms. *Physical Review A*, 107(3):032407, 2023.

- [132] Xiao-Hui Ni, Yu-Sen Wu, Bin-Bin Cai, Wen-Min Li, Su-Juan Qin, and Fei Gao. An adaptive mixer allocation strategy for the quantum alternating operator ansatz. *Advanced Quantum Technologies*, page e00487, 2025.
- [133] Akash Kundu, Aritra Sarkar, and Abhishek Sadhu. Kanqas: Kolmogorov-arnold network for quantum architecture search. *EPJ Quantum Technology*, 11(1):76, 2024.
- [134] Daochen Wang, Oscar Higgott, and Stephen Brierley. Accelerated variational quantum eigensolver. *Physical review letters*, 122(14):140504, 2019.
- [135] Rodrigo M Sanz, Andreu Angles-Castillo, Eduard Alarcon, and Carmen G Almudever. Efficiently architecting vqas: Expressibility–trainability–resources pareto-optimality. *arXiv preprint arXiv:2603.22142*, 2026.
- [136] Harsh Wadhwa, Rahul Bhowmick, Naipunnya Raj, Rajiv Sangle, Ruchira V Bhat, and Krishnakumar Sabapathy. Model selection in hybrid quantum neural networks with applications to quantum transformer architectures. *arXiv preprint arXiv:2603.21749*, 2026.
- [137] QIR Alliance. Quantum Intermediate Representation (QIR) specification, 2022.
- [138] Patrick Hopf, Erick Ochoa, Yannick Stade, Damian Rovara, Nils Quetschlich, Ioan Albert Florea, Josh Izaac, Robert Wille, and Lukas Burgholzer. Integrating quantum software tools with (in) mlir. In *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, pages 42–54, 2026.
- [139] Yannick Stade, Lukas Burgholzer, and Robert Wille. Towards supporting qir: Thoughts on adopting the quantum intermediate representation. *arXiv preprint arXiv:2411.18682*, 2024.
- [140] Alexander McCaskey and Thien Nguyen. A mlir dialect for quantum assembly languages. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 255–264. IEEE, 2021.
- [141] David Ittah, Thomas Häner, Vadym Kliuchnikov, and Torsten Hoefer. Qiro: A static single assignment-based quantum program representation for optimization. *ACM Transactions on Quantum Computing*, 3(3):1–32, 2022.
- [142] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. Compiler optimization for quantum computing using reinforcement learning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.
- [143] Amanda Xu, Abtin Molavi, Swamit Tannu, and Aws Albarghouthi. Optimizing quantum circuits, fast and slow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 777–793, 2025.
- [144] Arya Grayeli, Atharva Sehgal, Omar Costilla Reyes, Miles Cranmer, and Swarat Chaudhuri. Symbolic regression with a learned concept library. *Advances in Neural Information Processing Systems*, 37:44678–44709, 2024.
- [145] Christian Szegedy. A promising path towards autoformalization and general artificial intelligence. In *International Conference on Intelligent Computer Mathematics*, pages 3–20. Springer, 2020.
- [146] Yuhuai Wu, Albert Qiaoju Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. *Advances in neural information processing systems*, 35:32353–32368, 2022.
- [147] Yuanjie Ren, Jinzheng Li, and Yidi Qi. Merlean: An agentic framework for autoformalization in quantum computation. *arXiv preprint arXiv:2602.16554*, 2026.
- [148] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [149] Aritra Sarkar. Automated quantum software engineering. *Automated Software Engineering*, 31(1):36, 2024.